Swarm and Evolutionary Computation 102 (2026) 102338

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

Q-learning-based multi-objective hyper-heuristic algorithm for multi-campus university course scheduling

Zizhuang Zhu [a], Yongquan Zhou [a,c,*], Guo Zhou [b], Qifang Luo [a,c], Huajuan Huang [a,c]

[a] College of Artificial Intelligence, Guangxi Minzu University, Nanning 530006 China
[b] Department of Science and Technology Teaching, China University of Political Science and Law, Beijing 102249, China
[c] Guangxi Key Laboratories of Hybrid Computation and IC Design Analysis, Nanning 530006 China

ARTICLE INFO

Keywords:
University course scheduling problem
Multi-campus
Multi-objective optimization
Hyper-heuristic algorithm
Q-learning
Timetabling

ABSTRACT

The University Course Scheduling Problem (UCSP) is a critical combinatorial optimization challenge, particularly in multi-campus universities where cross-campus teacher commuting and resource coordination introduce unique complexities. Existing studies primarily focus on single-campus scenarios and single-objective optimization, failing to address the trade-offs between teacher commuting time, classroom utilization, and teaching quality. This study proposes a multi-objective optimization model (MCUCSP) for multi-campus university course scheduling problem explicitly incorporating cross-campus teacher commuting time, the number of classrooms utilized, and a comprehensive class hour index as key objectives. To solve this model, a Q-learning-based multi-objective hyper-heuristic algorithm (MO-QL-HH) is developed, which integrates high-level Q-learning and ten low-level heuristics (four random operators for exploration and six greedy operators for exploitation). Additionally, efficient initial solution generation (via greedy prioritization of high-difficulty tasks and classroom reutilization) and encoding-decoding mechanisms are designed to ensure constraint satisfaction and optimization efficiency. Systematic experiments on 20 multi-campus instances (2–6 campuses, 100–500 course tasks) demonstrate that MO-QL-HH outperforms well-known algorithms (NSGA-II, MOEA/D, NSGA-III, SPEA2, MOEA/D-AAWNs, and HGTSA) in terms of hypervolume (HV), inverted generational distance (IGD), and coverage metric (C-metric). Ablation experiments validate the necessity of integrating random and greedy operators, while parameter optimization via the full factorial design further enhances the algorithm's performance. This work provides an effective solution for multi-campus course scheduling, balancing teacher well-being, resource efficiency, and teaching quality.

1. Introduction

The timetabling problem is a type of combinatorial optimization problem that involves allocating a set of events with specific characteristics to limited resources under predefined constraints [1]. In real-life scenarios, numerous institutions are confronted with timetabling issues. For instance, schools face problems related to curriculum scheduling and exam arrangement [2,3]; hospitals deal with nurse rostering and surgery scheduling [4,5]; factories encounter worker scheduling and resource allocation challenges [6,7]; transportation systems have to handle the scheduling of bus and train timetables [8–10]; and various sports events involve competition schedule arrangement [11,12]. These all represent typical applications of the timetabling problem across diverse domains. The design and implementation of scheduling systems tailored to solve the timetabling problem are aimed at efficiently scheduling predefined activities (e.g., academic courses, medical procedures, and sports competitions) through coordinating the utilization of temporal availability and spatial capacity (e.g., educational facilities, operating rooms, and sports venues), while simultaneously optimizing the allocation of human resources (e.g., teachers, medical professionals, and event supervisors) [13]. Educational timetabling problems are generally divided into three main subtypes: High School Timetabling (HTT), University Course Timetabling (UCT), and University Exam Timetabling (UET) [14]. This study focuses on the university course scheduling problem.

Daskalaki et al. [15] described the university course scheduling problem as a resource allocation challenge that requires the temporal coordination of teaching arrangements within a weekly teaching cycle.

* Corresponding author at: College of Artificial Intelligence, Guangxi Minzu University, Nanning 530006 China.
E-mail address: zhouyongquan@gxmzu.edu.cn (Y. Zhou).

https://doi.org/10.1016/j.swevo.2026.102338
Received 9 November 2025; Received in revised form 28 December 2025; Accepted 12 February 2026
Available online 19 February 2026
2210-6502/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Research summary of UCSP.

Reference	Method	Objective type (single/multi-objective)	Multiple campuses	The scope of soft constraint optimization			
				Teacher	Class	Course	Classroom
[25]	Genetic Algorithm Integrated and Tabu Search(2024)	single	-	√		√	
[27]	Hybrid self-adaptive cuckoo search algorithm (2020)	single	-	√			√
[26]	Particle swarm optimization and selective search (2019)	single	-	√			
[43]	Particle swarm optimization and local search (2011)	single	-	√	√	√	
[1]	Competition-guided multi-neighborhood local search (2021)	single	-		√	√	√
[33]	Iterated local search using an add and delete hyper-heuristic (2016)	single	-		√	√	√
[44]	A simulated annealing based local search algorithm (2018)	single	-			√	
[45]	Genetic algorithm and local search algorithm (2014)	single	-		√	√	
[22]	Multi-objective mixed integer linear programming model (2023)	single	-	√	√	√	
[46]	multi-objective simulated annealing (2020)	multi (solution quality, solution robustness)	-		√	√	√
[47]	Tabu search and integer linear programming (2011)	single	-			√	√
[36]	Effective learning hyper-heuristics based on local search (2014)	single	-		√		
[15]	0–1 integer programming solvers (2004)	single	-	√	√		√
[19]	two-stage relaxation procedure (2005)	single	-	√	√		√
[48]	Adaptive Tabu Search algorithm (2010)	single	-			√	√
[49]	Genetic algorithm, simulated annealing and random search (2008)	single	-	√	√		
[50]	Artificial bee colony algorithm and great deluge algorithm (2015)	single	-		√		
[28]	Modified and hybridised bi-objective firefly algorithms (2023)	multi (operational cost, resource utilization)	-	√			√
[51]	Genetic algorithm and dynamic programming (2025)	single	-	√	√	√	√
[52]	Genetic algorithm and local search (2016)	single	-	√	√		√
[20]	Branch-and-Cut Procedure (2012)	single	-			√	√
[21]	Benders' Decomposition (2018)	single	-			√	√
[53]	Feature-based tuning of simulated annealing (2016)	single	-			√	√
This work	Q-learning-based multi-objective hyper-heuristic algorithm	multi (commuting time of cross-campus teachers, number of classrooms used, comprehensive class hour index)	√	√	√	√	√

In UCSP, the constraints of the problem are typically categorized into two types: mandatory (hard) constraints and preferential (soft) constraints. Hard constraints constitute non-negotiable prerequisites for scheduling, which originate from three operational dimensions: (1) the institutional teaching framework that governs the sequence of courses; (2) the physical resource configuration of teaching venues; (3) the time consistency requirements of teaching staff and student groups. Any violation of hard constraints will render the schedule structurally infeasible. In contrast, soft constraints represent preference-based optimization objectives, and the quality of the schedule is quantified through measurable violation indicators, which are usually formally expressed as weighted components in multi-objective optimization functions in mathematics [13]. Empirical studies have shown that there are significant differences in constraint priorities among different academic institutions [16]. Additionally, the course timetable can also have an impact on students' academic performance and sleep quality [17,18].

The solution approaches to the UCSP can be categorized into three types: exact algorithms, metaheuristic algorithms, and hyper-heuristic algorithms. Numerous scholars have employed exact algorithms to optimize the optimization models they established. For example, Daskalaki et al. [15,19] optimized the integer programming model they developed for the UCSP using a two-stage relaxation method and an integer programming solver; Burke et al. [20] constructed a branch-and-cut procedure for the Udine Course Timetabling problem; Bagger et al. [21] applied Benders' decomposition to the Curriculum-Based Course Timetabling problem; Mallari et al. [22] established a multi-objective mixed-integer linear programming model for the university course scheduling problem and optimized it using the CPLEX solver. Although exact algorithms theoretically guarantee global optimality through systematic search strategies, their practical applications are still limited to instances of small scale due to excessive resource demands [23]. As the problem scale expands, the UCSP grows at a non-polynomial rate, which fundamentally restricts the existence of exact solutions in polynomial time [24].

Computational complexity has driven the research focus toward metaheuristic algorithms, which balance solution quality and time efficiency through intelligent random exploration. When exploring complex solution spaces, metaheuristic algorithms have demonstrated effectiveness while maintaining practical computational costs. Academic literature has documented various innovative computational strategies for solving the UCSP. Xiang et al. [25] designed a novel two-stage metaheuristic algorithm: in the first stage, a genetic algorithm is used to cluster courses into sets via graph coloring techniques; in the second stage, an improved tabu search algorithm is employed to assign these clusters to different timeslots. Sk. Imran Hossain et al. [26] introduced a selective search mechanism into the particle swarm

optimization (PSO) algorithm to enhance the balance between exploration and exploitation, as well as to optimize the complexity of the search space. Song et al. [1] proposed a competition-oriented multi--neighborhood local search (CMLS) algorithm to address constraint problems in curriculum-based timetabling. Thepphakorn et al. [27] initially presented an adaptive hybrid cuckoo search algorithm to minimize the costs of educational institutions. This method was later extended via a bi-objective firefly algorithm (BOFA) incorporating the Pareto dominance principle, enabling simultaneous optimization of the financial efficiency and resource utilization of educational institutions [28].

Hyper-heuristics represent a general solution framework that adaptively selects optimizers to address complex optimization problems [29]. A hyper-heuristic framework consists of high-level selection strategies and low-level heuristic operators. Unlike metaheuristic methods that directly manipulate the solution space, hyper-heuristics do not operate directly on the solution space; instead, they indirectly manipulate it by managing a set of low-level heuristic operators [30,31]. This paradigm shift enables heuristic methods to dynamically adapt during the problem-solving phase, optimizing the algorithm's responsiveness to complex constraints [32]. Hyper-heuristic algorithms have been practically applied to the UCSP. For instance, Soria-Alcaraz et al. [33] proposed an add-and-delete hyper-heuristic iterative local search algorithm to optimize the university course timetabling problem. Muklason et al. [34] developed a hyper-heuristic algorithm incorporating tabu search (TS) and variable neighborhood search (VNS) to solve the UCSP, aiming to generate highly flexible optimal solutions. Kalender et al. [35] adopted a hyper-heuristic algorithm based on greedy gradient simulated annealing for UCSP scheduling tasks. Additionally, Soria-Alcaraz et al. [36] implemented an iterative local search hyper-heuristic framework that incorporates a learning mechanism and multiple move operators to handle course timetabling problems.

As one of the successful strategies in reinforcement learning, Q-learning enables an agent to determine optimal behaviors by learning an action-value function, which represents the expected utility of applying an appropriate action under specific states [37]. Researchers have extensively studied and utilized Q-learning to address the problem of how agents make decisions in complex scenarios [38]. In general, Q-learning trains the agent (via a Q-table) using Monte Carlo methods; subsequently, the agent obtains expected Q-values between different states based on the trained Q-table and its current states, thereby making judgments [39]. Therefore, Q-learning-based hyper-heuristic algorithms can provide appropriate high-level selection strategies to choose suitable search behaviors and guide the search trend toward promising regions.

In Table 1, we present the different optimization methods employed in existing literature, along with the types of objective functions of the UCSP model and the optimization scope of soft constraints. For more detailed information and descriptions of the UCSP, readers are referred to references [2,40–42].

In real-world scenarios, with the development of human society and the increasing number of students, many universities have expanded from a single campus to multiple campuses. Due to insufficient teacher resources, many teachers need to commute between different campuses, which can cause physical and mental fatigue and stress for these cross-campus teachers, thereby leading to a decline in teaching quality. However, existing research on UCSP has focused entirely on single campuses, neglecting the optimization of commuting frequency and time for these cross-campus teachers during the teaching cycle. This study focuses on joint course scheduling across multiple campuses of the same university, while comprehensively considering the connections and optimization among multiple dimensions including classes, teachers, courses, and classrooms. It aims to achieve people-oriented course arrangements, optimize the commuting time of cross-campus teachers, reduce the work pressure of teachers and the academic burden of students, meet the teaching time preferences of different teachers, complete course arrangements with fewer classrooms, and optimize resource utilization.

Meanwhile, existing studies have mostly focused on single-objective UCSP models, that is, most of them use the weighting method to convert multiple soft constraints into a single objective for optimization. Although this method simplifies the optimization model, it is actually imperfect. Specifically, firstly, the setting of weight coefficients often relies on subjective experience and judgment, lacking objective quantitative basis. Differences in preferences among different decision-makers may lead to significant deviations in weight allocation, thereby causing the optimization results to lose universality. Secondly, there are often non-linear conflicting relationships among multiple soft constraint objectives (e.g., the trade-off between teachers' teaching time preferences and classroom resource utilization). Weighted summation will mask the inherent trade-off mechanisms among objectives, making it difficult to generate non-dominated solutions that fully reflect the optimization level of each objective. Thirdly, this method will compress the objective space dimension, resulting in the loss of original constraint information. When the priority of soft constraints is dynamically adjusted, the model lacks flexible adaptability and may produce optimization results that contradict actual needs. This study adopts multi-objective optimization models and methods, which can make up for these deficiencies, thereby enabling the optimization results to more comprehensively retain the original information of each soft constraint objective, objectively present the trade-off relationships among objectives, and simultaneously possess flexible adaptability to cope with dynamic changes in constraint priorities, thus enhancing the practical application value of the solutions.

The aforementioned studies, while demonstrating the applicability of hyper-heuristic algorithm to timetabling, primarily focus on single-campus and often single-objective settings. The MCUCSP introduces distinct challenges due to its multi-campus nature (e.g., cross-campus teacher commuting) and inherent multi-objective trade-offs (e.g., teacher welfare vs. resource efficiency). Addressing these challenges requires optimization methods that can effectively navigate a complex constraint landscape and balance conflicting objectives. In this context, hyper-heuristic algorithm offer a significant advantage through their separation of concerns: a high-level strategy dynamically selects or combines low-level heuristics tailored to different search stages or problem characteristics. This adaptability is crucial for MCUCSP. Furthermore, integrating learning mechanisms, particularly reinforcement learning like Q-learning, into the high-level strategy has proven highly effective in other complex scheduling domains. For example, knowledge-enhanced hyper-heuristic evolutionary algorithms have successfully tackled semiconductor manufacturing scheduling problems characterized by high dimensionality and multiple constraints [61]. Similarly, Q-learning-based multi-objective hyper-heuristic algorithm have shown superior performance in energy-efficient distributed hybrid flow shop scheduling [62] and distributed flexible job-shop scheduling [63], problems that share with MCUCSP the complexities of multi-objective optimization, resource distribution, and combinatorial explosion. These successes underscore the potential of Q-learning-enhanced hyper-heuristic algorithm to manage the exploration-exploitation dilemma and adaptively select heuristics for the multifaceted MCUCSP.

However, to the best of the authors' knowledge, the integration of Q-learning within a hyper-heuristic framework has not yet been explored for the UCSP, especially in a multi-campus, multi-objective context. This study combines Q-learning with hyper-heuristic algorithms and designs 10 heuristic operators based on the characteristics of the problem to further improve the application of hyper-heuristic algorithms in UCSP.

The contributions of this paper are as follows:

(1) A novel multi-campus multi-objective university course scheduling model (MCUCSP) is constructed, in which the commuting time of cross-campus teachers is explicitly incorporated as a key

Table 2
Notation definitions.

Sets	
I	Set of task indices
J	Set of teacher indices
K	Set of classroom indices
L	Set of campus indices
C	Set of class indices
S	Set of course indices
T	Set of timeslot indices in a week ($T = \{1,2,3,4,5\}$, with 5 timeslots per day)
D	Set of weekday indices ($D = \{1,2,3,4,5\}$, from Monday to Friday)
W	Set of week indices ($W = \{1,2, \ldots,16\}$, totaling 16 weeks)
UD_j	Set of unavailable weekdays in a week for teacher $j \in J$
UT_j	Set of unavailable timeslots in a day for teacher $j \in J$
UT_i	Set of unavailable timeslots in a day for task $i \in I$
CI	Set of tasks with time scheduling constraints
JI	Set of course tasks taught by teachers with time scheduling constraints
I_j	Set of course tasks to be taught by teacher $j \in J$
I_c	Set of course tasks to be attended by class $c \in C$
I_s	Set of all course tasks corresponding to course $s \in S$
L_j	Set of campuses where cross-campus teacher $j \in J$ can teach
CJ	Set of cross-campus teachers
Parameters	
t_i	Type of task $i \in I$
N_i	Number of class hours of task $i \in I$
ct_k	Type of classroom $k \in K$
cap_k	Capacity of classroom $k \in K$
d_i	Number of classes corresponding to task $i \in I$
l_i	Campus to which task $i \in I$ belongs
Lc	Campus to which class $c \in C$ belongs
l_k	Campus to which classroom $k \in K$ belongs
$c_{j,l}$	Commuting time of teacher $j \in J$ to campus $l \in L$
PD_j	Preference weight of teacher $j \in J$ for different weekdays in a week, $PD_j \in \{-1,0,1\}$.
PT_j	Preference weight of teacher $j \in J$ for different timeslots in a day, $PT_j \in \{-1,0,1\}$.
PT_s	Appropriateness weight of course $s \in S$ for different timeslots in a day, $PT_s \in \{-1,0,1\}$.
Variables	
$x_{i,l,k,w,d,p}$	Takes the value 1 if task i is assigned to be taught in campus l, classroom k, week w, weekday d, and timeslot t; otherwise, 0
u_k	Takes the value 1 if classroom $k \in K$ is used; otherwise, 0
$y_{j,l,w,d}$	Takes the value 1 if teacher j teaches in campus l in week w and weekday d; otherwise, 0
z_i	$z_i = 0$ if the number of timeslots assigned to course task i in a day does not exceed 1; otherwise, z_i equals the number of excess timeslots.

optimization objective alongside classroom usage and a comprehensive class hour index. Both hard constraints (e.g., prohibiting cross-campus teachers from teaching at different campuses on the same day) and soft constraints (e.g., teacher time preferences) are integrated, thereby effectively addressing the limitations of existing single-campus and single-objective formulations.

(2) A Q-learning-based multi-objective hyper-heuristic algorithm (MO-QL-HH) is proposed, which innovatively combines high-level Q-learning with ten low-level heuristics (four random operators for global exploration and six greedy operators for local exploitation). This design enables dynamic adaptation to the search state, effectively balancing exploration and exploitation to enhance optimization performance in the complex MCUCSP landscape.

(3) Efficient initial solution generation and encoding-decoding mechanisms tailored for multi-campus scheduling are designed. The greedy initial solution strategy prioritizes high-difficulty tasks (via difficulty scores) and promotes classroom reuse to reduce fragmentation. The structured encoding-decoding ensures constraint satisfaction and provides a solid foundation for subsequent optimization.

(4) Through systematic experiments on 20 multi-campus instances, the superiority of MO-QL-HH is demonstrated over several well-known algorithms (NSGA-II, MOEA/D, NSGA-III, SPEA2, MOEA/D-AAWNs, and HGTSA) in terms of HV, IGD, and C-metric. The necessity of integrating both random and greedy operators is validated by ablation experiments, while parameter optimization via full factorial design is shown to further enhance algorithmic performance.

The remaining content of this paper is organized as follows: Section 2 provides a detailed description of the studied MCUCSP and establishes a multi-objective optimization mathematical model for the problem. Section 3 elaborates on the details of the high-level Q-learning strategy and low-level heuristic operators of MO-QL-HH. Section 4 systematically presents the experimental settings, computational results, and performance comparison evaluations. Section 5 summarizes the entire paper and prospects future research contents and directions.

2. Problem description and mathematical model

2.1. Problem description

Unlike the single-campus scenario, the course scheduling problem involving multiple campuses requires consideration of the commuting time for cross-campus teachers to ensure they are not frequently shuttling between different campuses. Classrooms across different campuses are generally used independently, meaning that course tasks from one campus cannot be scheduled in classrooms of another campus. Within a teaching cycle, we divide the cycle into multiple teaching weeks. In each teaching week, we assume there are five working days (Monday to Friday) available for course scheduling, and each working day can be further divided into five timeslots: two in the morning, two in the

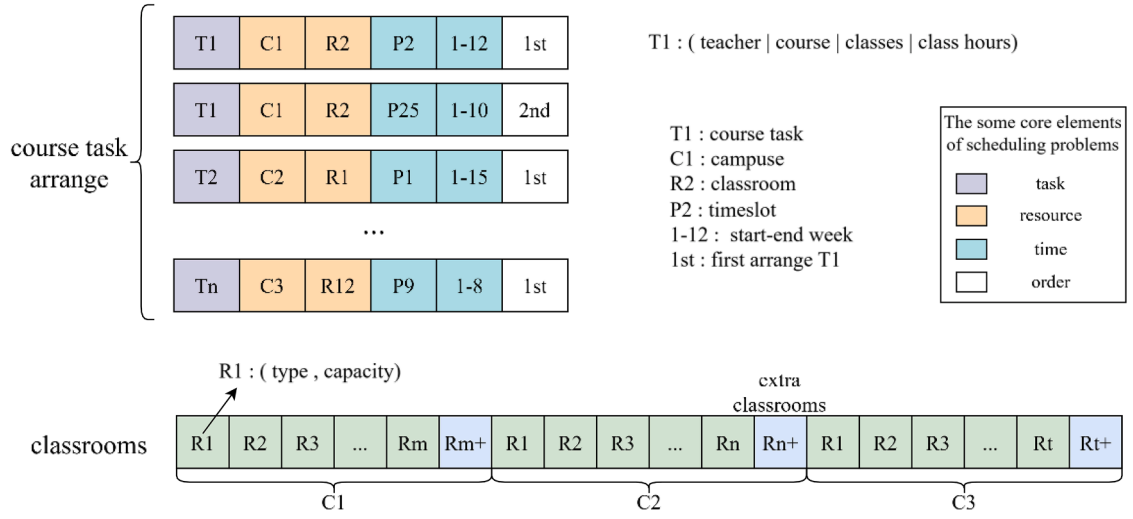


Fig. 1. Coding scheme for solutions.

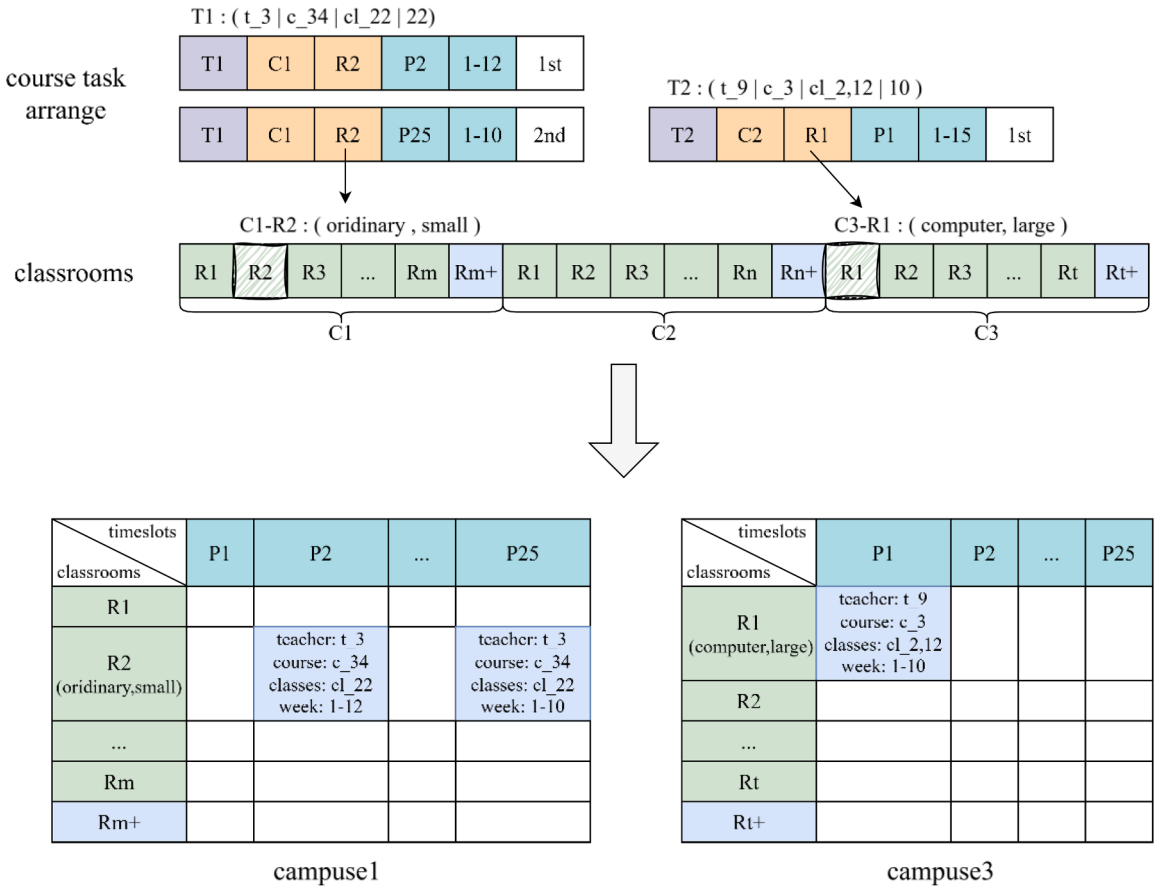


Fig. 2. Decoding scheme for solutions.

afternoon, and one in the evening. To ensure the rationality and practicality of the course schedule, we have considered the following hard constraints and soft constraints.

Hard Constraints:

(1) A single class can only be scheduled for one course at the same time.
(2) A single teacher can only teach one course at the same time.
(3) A single classroom can only be assigned to one course at the same time.
(4) The course type must match the classroom type.
(5) The classroom capacity must meet the course requirements.
(6) Teachers cannot be scheduled for courses during unavailable weekdays or timeslots.
(7) Courses cannot be scheduled during their unavailable timeslots.
(8) Teachers who commute across campuses cannot be assigned courses at different campuses on the same day.

Algorithm 1
The process of generating the initial solution based on the greedy strategy.

Input: *CT* (course tasks), *CP* (campuses), *TR* (teachers), *CL* (classrooms)
Output: *SL* (schedule list)
Initialize $SL \leftarrow \emptyset$;
// **Greedy Strategy 1**: Prioritize arranging course tasks with higher difficulty scores
Compute difficulty score $DS(c) \leftarrow$ number of classes (c) $\times$ class hours (c) $\forall c \in CT$;
Sort $CT \leftarrow$ descending (DS);
Initialize $UC \leftarrow \emptyset$; // used classrooms
For each course task c in CT **do**
  $sessions \leftarrow$ The number of weekly arrangements for course task c;
  While $sessions > 0$ **do**
    $SC \leftarrow$ The set of classrooms suitable for course task c;
    If SC $= \emptyset$ **then**
      $SC \leftarrow$ alternative classrooms that meet the requirements of course task c;
    End If
    // **Greedy Strategy 2**: Prioritize the use of classrooms that have been used
    Prefer $R \in SC$ subject to R.name $\in UC$; // reuse used classrooms greedily
    For each timeslot T in shuffled (all timeslots) **do**
      If no conflict (course task c, timeslot T, schedule list SL) **then**
        $SL \leftarrow SL \cup$ {schedule (course task c, classroom R, timeslot T)};
        $UC \leftarrow UC \cup \{R.\text{name}\}$;
        $sessions \leftarrow sessions - 1$;
        Break
      End If
    End For
  End While
End For
Return *SL*;

(9) Course tasks must be scheduled only in the campus they belong to.
(10) All class hours of course tasks must be fully scheduled.

Soft Constraints:

(1) The commuting time of cross-campus teachers should be minimized.
(2) Course tasks should be arranged more frequently during daytime.
(3) The number of classrooms used in course scheduling should be minimized.
(4) Teachers' teaching schedules should be arranged as much as possible during their preferred weekdays and timeslots.
(5) Courses should be scheduled at their appropriate timeslots as much as possible.
(6) Excessive sessions of the same course should not be arranged for a single class on the same day.

Hard constraints ensure that the results of course scheduling do not violate the objective requirements in the real world. The design of soft constraints enables a people-oriented course arrangement: (1) It optimizes and reduces the commuting time of cross-campus teachers, preventing them from shuttling back and forth between different campuses. (2) It reduces the excessive arrangement of course tasks, so as to alleviate the work pressure on teachers and the academic burden on students. (3) It meets the teaching time preferences of different teachers (e.g., teachers can select their preferred teaching timeslots). (4) It arranges various types of courses in more appropriate time periods (e.g., physical education courses are optimally scheduled in the afternoon). (5) Ultimately, it completes the course arrangement using as few classrooms as

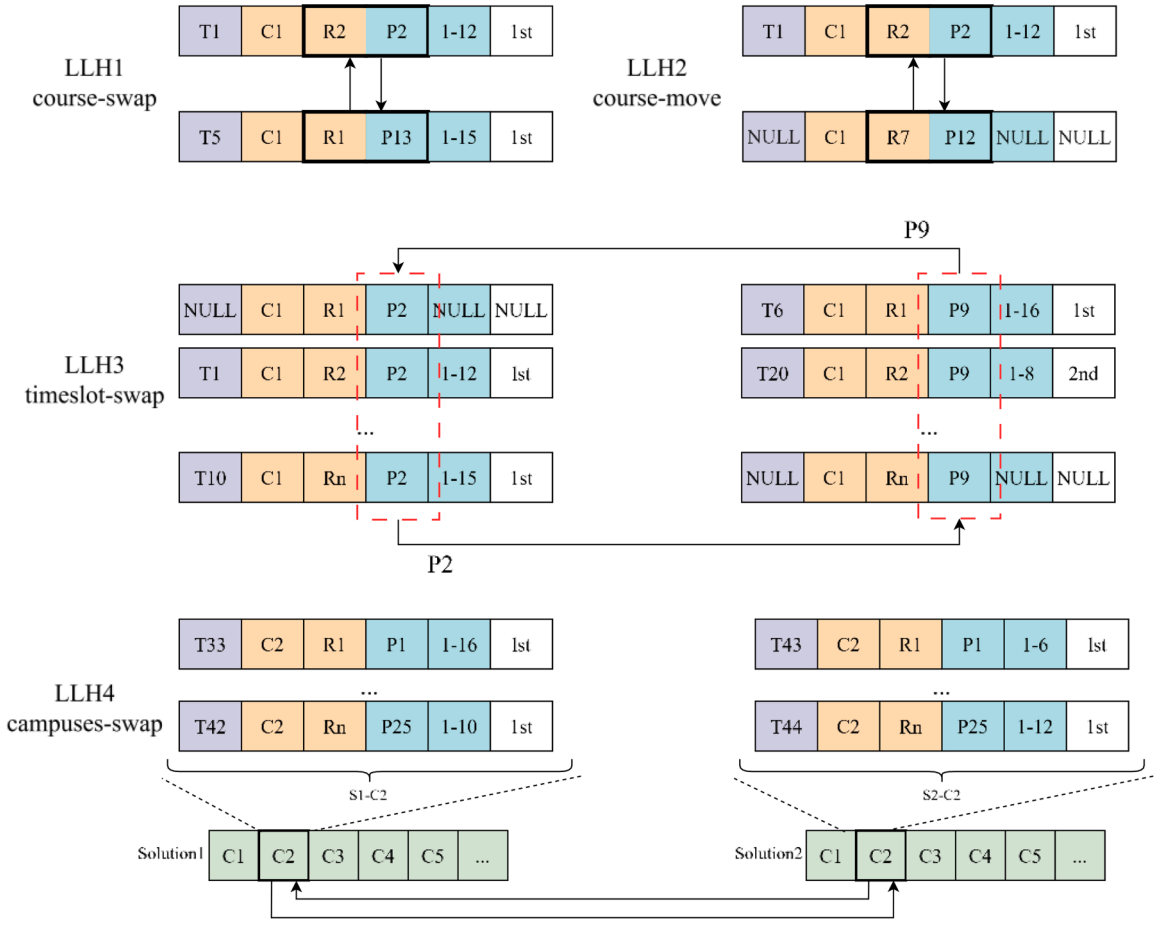


Fig. 3. Four random operators.

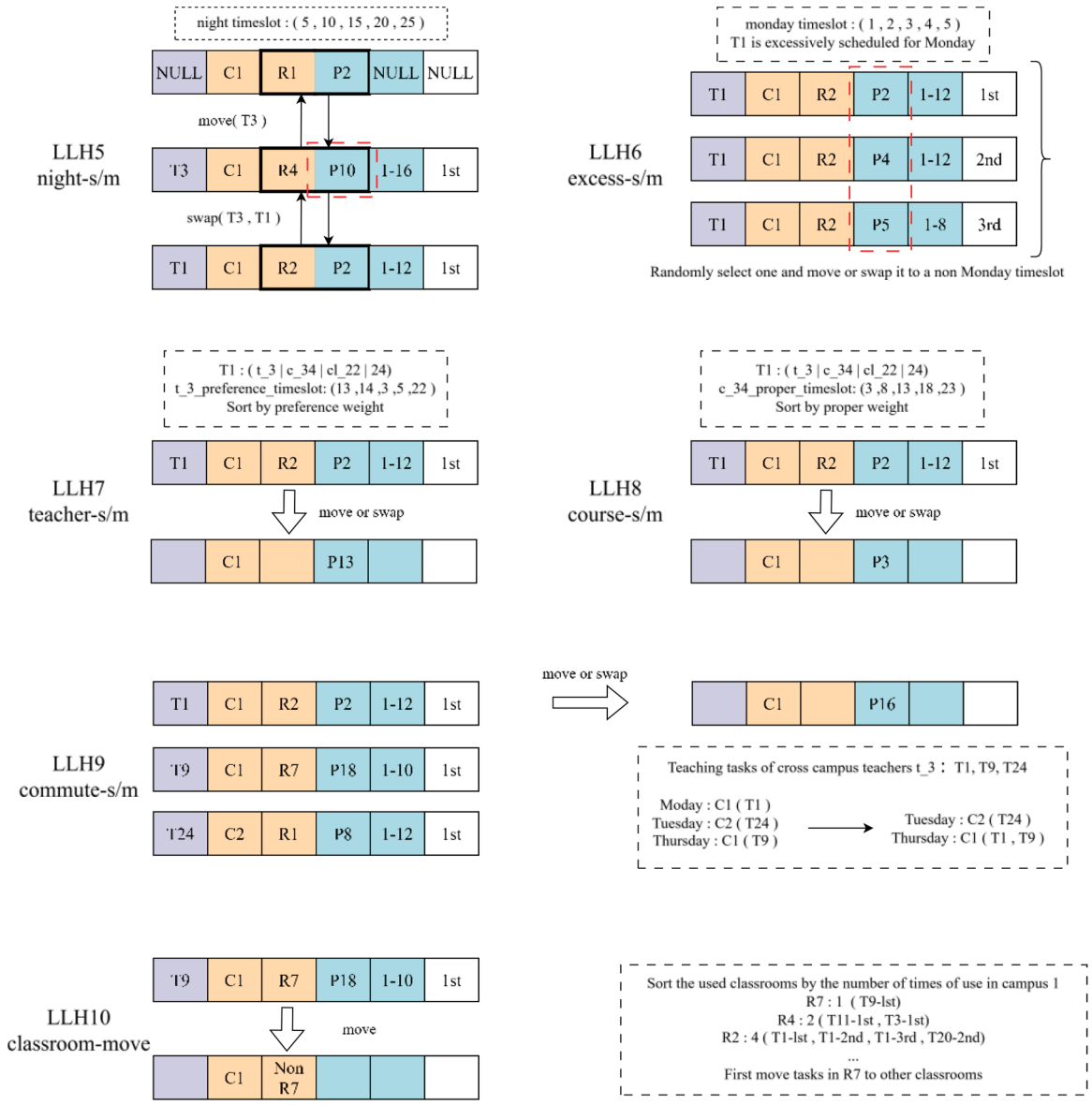


Fig. 4. Six greedy operators.

Algorithm 2
Low-Level Heuristic (LLH) Operator General Process.

Input: S (Current solution), N (The selected operator serial number)
Output: S' (Modified solution)
1: Initialize $S' \leftarrow S$; // Create a copy of the current solution
// **Step 1**: Perform LLH operator
2: Select the LLH operator Op to be executed from the LLH operator set according to N;
3: Apply operator Op to solution S' to generate a modified solution S'';
// **Step 2**: Repair the solution
4: Initialize repair list $RL \leftarrow \emptyset$; // List to store repair steps
5: **For** each constraint violation V in S'' **do**
6: Determine the type of constraint violation Vt;
7: Select a repair strategy RS based on Vt;
8: Apply repair strategy RS to fix violation V in S'';
9: $RL \leftarrow RL \cup$ {repair step taken};
10: **End For**
11: Update S' with the repaired solution S'';
12: **Return** S';

possible, thereby optimizing resource utilization.

2.2. Mathematical model

Based on the characteristics of the problem, as well as the hard constraints and soft constraints, we have established a multi-objective optimization mathematical model for this problem, which is presented as follows.

2.2.1. Notation definitions

The notation definitions for this mathematical model are presented in Table 2.

2.2.2. Hard constraints

1. Constraint (1) ensures that a single class can only attend one course at the same time.

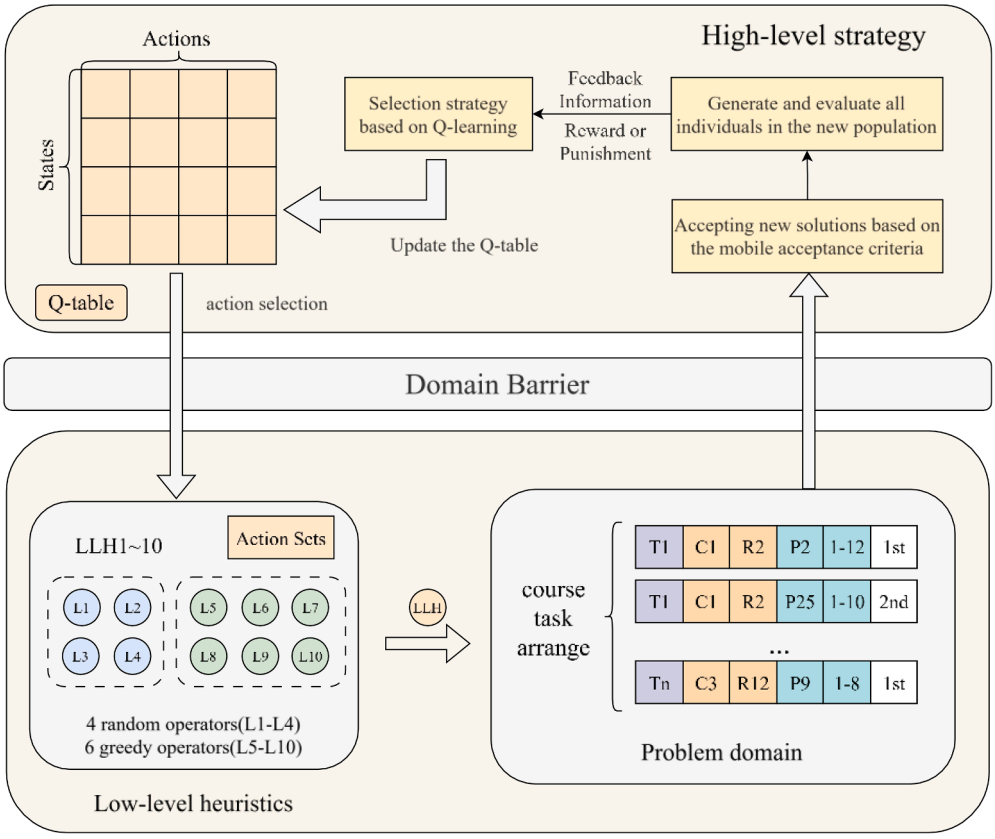


Fig. 5. Framework diagram of the MO-QL-HH.

$$\sum_{i\in I}\sum_{k\in K} x_{i,l_c,k,w,d,t} \le 1 \forall c \in C, w \in W, d \in D, t \in T \quad (1)$$

2. Constraint (2) ensures that a single teacher can only teach one course at the same time.

$$\sum_{i\in I}\sum_{l\in L_j}\sum_{k\in K} x_{i,l,k,w,d,t} \le 1 \forall j \in J, w \in W, d \in D, t \in T \quad (2)$$

3. Constraint (3) ensures that a single classroom can only be assigned to one course at the same time.

$$\sum_{i\in I} x_{i,l_k,k,w,d,t} \le 1 \forall k \in K, w \in W, d \in D, t \in T \quad (3)$$

4. Constraint (4) ensures that the course type matches the classroom type.

$$x_{i,l_i,k,w,d,t} = 0 \text{if} t_i \ne ct_k \forall i \in I, k \in K, w \in W, d \in D, t \in T \quad (4)$$

5. Constraint (5) ensures that the classroom capacity meets the course requirements.

$$x_{i,l_i,k,w,d,t} = 0 \text{if} cap_k < d_i \forall i \in I, k \in K, w \in W, d \in D, t \in T \quad (5)$$

6. Constraints (6)-(7) ensure that teachers cannot be scheduled for courses during their unavailable weekdays or timeslots.

$$x_{i,l_i,k,w,d,t} = 0 \forall i \in JI, k \in K, w \in W, d \in UD_j, t \in T \quad (6)$$

$$x_{i,l_i,k,w,d,t} = 0 \forall i \in JI, k \in K, w \in W, d \in D, t \in UT_j \quad (7)$$

7. Constraint (8) ensures that courses cannot be scheduled during their unavailable timeslots.

$$x_{i,l_i,k,w,d,t} = 0 \forall i \in CI, k \in K, w \in W, d \in D, t \in UT_i \quad (8)$$

8. Constraint (9) ensures that cross-campus teachers cannot be assigned courses at different campuses on the same day.

$$\sum_{l_1\in L_j}\sum_{l_2\in L_j, l_1\ne l_2} y_{j,l_1,w,d} \times y_{j,l_2,w,d} = 0 \forall j \in Cj, w \in W, d \in D \quad (9)$$

9. Constraint (10) ensures that course tasks can only be scheduled within their affiliated campus.

$$x_{i,l_i,k,w,d,t} = 0 \text{if} l \ne L_i \forall i \in I, k \in K, w \in W, d \in D, t \in T \quad (10)$$

10. Constraint (11) ensures that all class hours of course tasks must be fully scheduled.

$$\sum_{k\in K}\sum_{w\in W}\sum_{d\in D}\sum_{t\in T} x_{i,l,k,w,d,t} = N_i \forall i \in I \quad (11)$$

2.2.3. Soft constraints

1. The commuting time from the residence of each cross-campus teacher to different associated campuses varies. Eq. (12) calculates the total commuting time for all cross-campus teachers.

$$f_1 = \sum_{j\in CJ}\sum_{l\in L_j}\sum_{w\in W}\sum_{d\in D} y_{i,l,w,d} \times c_{j,l} \quad (12)$$

2. Classrooms in each campus are used independently. Eqs. (13)-(14) calculate the number of classrooms used across all campuses.

$$u_k = \begin{cases} 1, & \text{if} \sum_{i\in I}\sum_{w\in W}\sum_{d\in D}\sum_{t\in T} x_{i,l_i,k,w,d,t} > 0 \\ 0, & \text{otherwise} \end{cases} \quad (13)$$

$$f_2 = \sum_{k\in K} u_k \quad (14)$$

3. Efforts are made to schedule fewer course tasks in the evening and more during daytime. The 5th timeslot of each day corresponds to the evening. Eq. (15) calculates the total class hours of course tasks scheduled in the evening across all campuses.

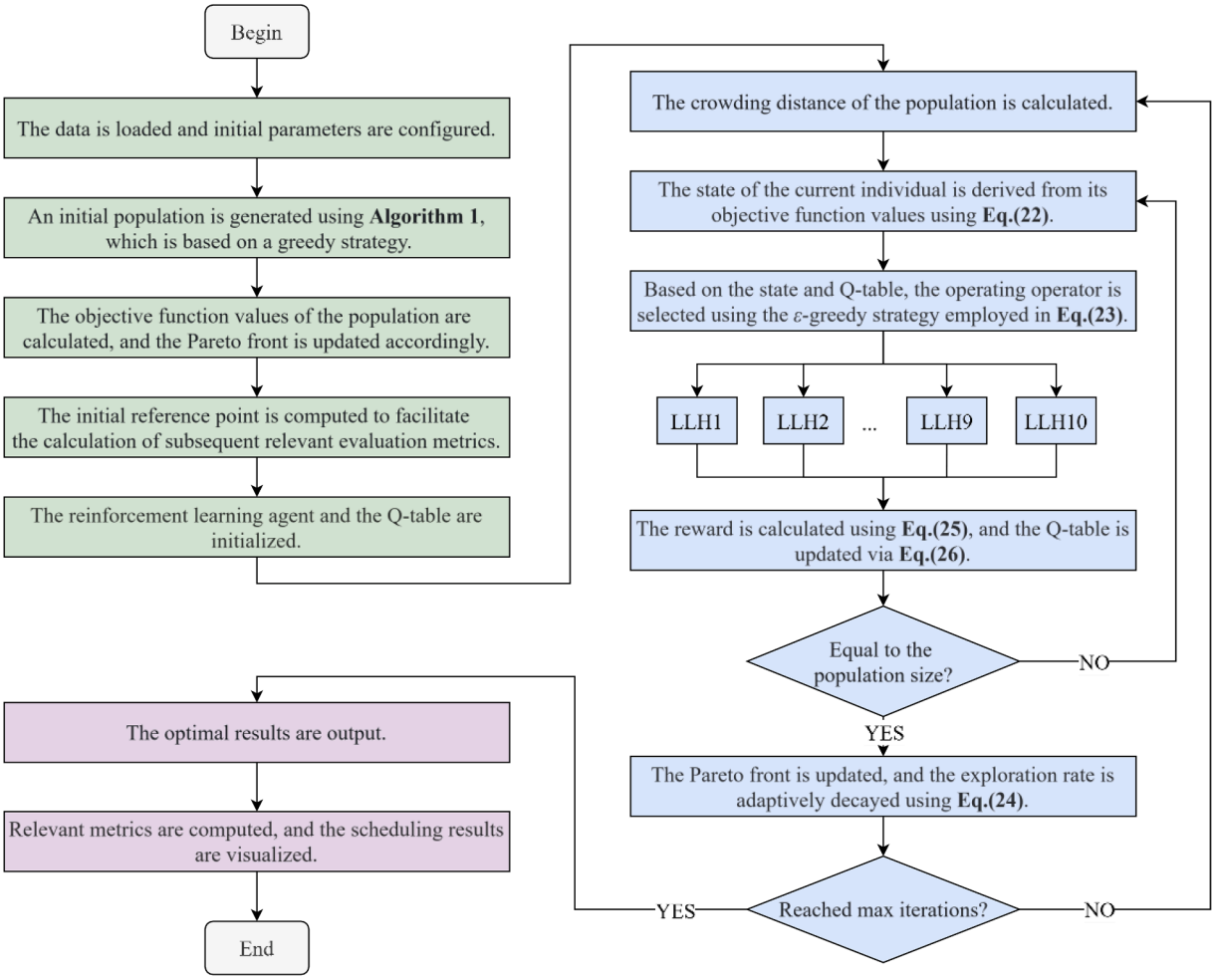


Fig. 6. Flowchart of the MO-QL-HH.

$$f_{3a} = \sum_{i\in I}\sum_{k\in K}\sum_{w\in W}\sum_{d\in D} x_{i,l_i,k,w,d,5} \tag{15}$$

4. Teachers' teaching tasks should be arranged as much as possible during their preferred weekdays and timeslots. Eq. (16) calculates the total class hours taught by all teachers within their preferred timeslots.

$$f_{3b} = \sum_{j\in J}\sum_{i\in I_j}\sum_{k\in K}\sum_{w\in W}\sum_{d\in D}\sum_{t\in T} x_{i,l_i,k,w,d,t} \times \left(PT_j + PD_j\right) \tag{16}$$

5. Courses should be scheduled in their appropriate timeslots as much as possible. Eq. (17) calculates the total class hours of all courses arranged in their appropriate timeslots.

$$f_{3c} = \sum_{s\in S}\sum_{i\in I_s}\sum_{k\in K}\sum_{w\in W}\sum_{d\in D}\sum_{t\in T} x_{i,l_i,k,w,d,t} \times PT_s \tag{17}$$

6. For the same course of a given class, the number of class hours scheduled on the same day should not exceed one timeslot. Eqs. (18)-(19) calculate the total number of excessive class hours across all courses.

$$z_i = \begin{cases} \sum_{k\in K}\sum_{t\in T} x_{i,l_i,r,w,d,t} - 1, & \text{if} \sum_{k\in K}\sum_{t\in T} x_{i,l_i,k,w,d,t} > 1 \\ 0, & \text{otherwise} \end{cases} \tag{18}$$

$$f_{3d} = \sum_{c\in C}\sum_{w\in W}\sum_{d\in D}\sum_{i\in I_c}\left(\frac{z_i \times N_i}{z_i + 1}\right) \tag{19}$$

2.2.4. Objective function

The objective function of the MCUCSP consists of three components: the commuting time of cross-campus teachers (*TT*, in minutes), the number of classrooms used (*CU*, in units), and the comprehensive class hour index (*CCI*, in class hours), as shown in Eq. (20).

$$\begin{aligned} &\text{objective}: \min\{TT, CU, CCI\} \\ &\begin{cases} TT = f_1 \\ CU = f_2 \\ CCI = f_3 = f_{3a} + f_{3d} - f_{3b} - f_{3c} \end{cases} \end{aligned} \tag{20}$$

3. The proposed MO-QL-HH

3.1. Encoding and decoding schemes

The encoding mechanism converts scheduling schemes into structured computable sequences through multi-dimensional symbolic mapping, thereby fully representing the solution space. For each course task, six core elements form an encoding tuple: task identifier, campus identifier, classroom identifier timeslot index, execution period, and arrangement order Classroom identifiers embed attribute information, while timeslots are converted into 0-based indices ranging from 1 to 25. The final encoding takes the form of an ordered tuple list, which retains core constraints and scheduling logic, thus providing structured input for optimization operators. The specific encoding scheme is illustrated in Fig. 1.

As the inverse process of encoding, the decoding mechanism

reconstructs symbolic sequences into executable scheduling schemes and verifies constraint satisfaction. It first matches actual campuses and classrooms using their respective identifiers and checks attribute compatibility. Subsequently, it locates course tasks via task id to extract associated parameters such as the teacher and class, parses timeslot indices into specific periods, determines the full time range by integrating execution weeks, and finally handles multi-instance scheduling in arrangement order to avoid conflicts. Multi-layer verification is implemented to ensure compliance with constraints, such as non-overlapping timeslots for teacher and matching between classroom capacity and class size. The output adheres to hard constraints, supporting objective function calculation and optimization performance evaluation. The specific decoding scheme is illustrated in Fig. 2.

3.2. The initial solution generation strategy based on the greedy strategy

This study employs a greedy strategy-based algorithm for initial solution generation (Algorithm 1), optimizing scheduling efficiency through a dual mechanism: (1) Task Prioritization: Tasks are dynamically sorted by scheduling difficulty, quantified by the difficulty score DS (Eq. (21)). High-difficulty tasks (e.g., large-scale classes or extended-duration courses) are prioritized through descending DS ordering, significantly reducing backtracking risks associated with highly constrained tasks. (2) Classroom Reuse: Subject to classroom type and capacity constraints, previously allocated classrooms are preferentially selected (90 % probability weight). This strategy maintains a global set of utilized classrooms, minimizing resource fragmentation while suppressing temporary classroom creation.

These strategies operate synergistically during timeslot allocation: Randomized timeslot traversal dynamically validates teacher-time conflicts, class-time conflicts, and cross-campus mobility constraints for each task's weekly session demand. Iteration continues until all tasks are scheduled or the retry threshold is reached.

$$DS(c) = L_c \times D_c \tag{21}$$

where L_c denotes the number of classes simultaneously attending course task c, and D_c denotes the number of course hours for course task c.

3.3. Q-learning-based high-level strategy

The high-level policy of MO-QL-HH is formulated as a Markov Decision Process (MDP) to adaptively guide low-level operators in multi-objective optimization scenarios. Its core components, designed to balance exploration and exploitation while optimizing multiple objectives, are elaborated as follows:

(1) **State Definition**: The state is defined by discretizing the normalized objective values of the current solution. This design not only reduces the dimensionality of the state space, but also retains the key characteristics of the solution's performance in the objective space. Each objective value is scaled by a reference point r and discretized into integers between 0 and 3, the calculation formula is shown in Eq. (22).

$$s = \left(\left\lfloor \frac{f_1}{r_1} \times 3 \right\rfloor, \left\lfloor \frac{f_2}{r_2} \times 3 \right\rfloor, \ldots, \left\lfloor \frac{f_m}{r_m} \times 3 \right\rfloor \right) \tag{22}$$

where s represents the discretized state vector, which is composed of the integer scaled and discretized objective values of the current solution; f_i denotes the i th objective value of the current solution (a minimization target); r_i is the i th component of reference point r, used to normalize f_i and eliminate scale differences between objectives; m represents the total number of objectives; and scaling by 3 maps normalized values (ranging from 0 to 1) to integers between 0 and 3 for state discretization, reducing dimensionality while preserving critical solution characteristics.

(2) **Action Definition**: To balance global exploration and local refinement, the action set $A=\{a_0,\ldots,a_9\}$ is explicitly partitioned into two categories, as shown in Figs. 3 and 4. Random operators $A_{\text{rand}}=\{a_0,\ldots,a_3\}$ include course swap, course move, campus swap, and timeslot swap, introducing large-scale stochastic perturbations to enhance population diversity. Greedy operators $A_{\text{greedy}}=\{a_4,\ldots,a_9\}$ include night adjust, class adjust, teacher preference, course period, reduce commute, and reduce classroom, performing heuristic-guided local improvements to accelerate convergence to the Pareto front. This taxonomy allows adaptive switching between exploration and exploitation based on the search stage.

(3) **Action-selection strategy**: Actions are selected via an ε-greedy rule to balance exploration (discovering new solutions) and exploitation (refining known good solutions). When the probability is ε, it explores random actions, and when the probability is $1-\varepsilon$, it performs the current

Algorithm 3
The proposed MO-QL-HH algorithm.

Input: *PS* (population size), *MI* (max iterations), *LR* (learning rate), *DF* (discount factor), *ER* (exploration rate), *ED* (exploration decay), *RP* (reference point)
Output: *PF* (pareto front), *HV* (final hypervolume), *AT* (RL agent)
Initialize population $P = \emptyset$ and Pareto front $PF = \emptyset$;
For $n = 1$ to *PS* **do**
  Generate initial schedule and campuses using Algorithm 1;
  Encode schedule to get encoded schedule;
  Create tuple (encoded schedule, campuses) as an individual of population *P*;
Update pareto front *PF*;
End For
Initialize *AT* and Q-table $QT \in \mathbb{R}^{\wedge}(|S| \times |A|)$ ($|S|$: state space size, $|A|$: action space size);
For *iter* $= 1$ to *MI* **do**
  Calculate crowding distance of population *P*;
  Calculate current hypervolume *HV* using *PF, RP* and Eq. (29);
  Initialize new population $NP = \emptyset$;
  For each individual $CI \in P$ **do**
    Extract state $S=(f_1(CI), \ldots, f_k(CI))$ by Eq. (22), where f_m is the *m*-th objective function;
    Select the action *A* to be performed by *CI* using Eq. (23);
    Execute specific operator *OA* using Algorithm 2 on *CI* based on action *A*;
    If operator *OA* executes successfully and *NI* is valid **then**
      Add *NI* to *NP*;
      Calculate crowding distance of *NP*;
      Calculate the reward of the state-action pair (*S*-*A*) using Eq. (25);
      Update the Q-table *QT* using *LR, DF* and Eq. (26);
      Update *PF* by checking dominance of *NI* over existing *PF* members;
      If the *PF* is not successfully updated **then**
        If random (0,1) $<$ *ER* **then**
          NP [end] $=$ *NI* (replace last individual in *NP* with *NI*);
        Else If $PF \neq \emptyset$ **then**
          NP [end] $=$ *rand* (*PF*) (randomly select from *PF*);
        End If
      End If
    Else
      Add *CI* to new population *NP* (retain parent if operator fails);
    End If
  End For
  Set population $P =$ new population *NP*;
  Decay the exploration rate *ER* using *HV, ED* and Eq. (24);
End For
Compute final hypervolume *HV* using *PF, RP* and Eq. (29);
Return pareto front *PF*, final hypervolume *HV*, RL agent *AT*;

Table 3
Summary of the time complexity and space complexity of MO-QL-HH.

Type	Component	Complexity
Time	Initialization	$O(N \times C \times (\log C + P \times R))$
	Fitness Evaluation	$O(N \times C \times (P + A + B))$ per iteration
	Q-Learning	$O(N \times K)$ per iteration
	LLH Operators	$O(N \times P^2 \times R)$ per iteration
	Total	$O(N \times C \times (\log C + P \times R) + T \times N \times (K + C \times (P + A + B) + P^2 \times R))$
Space	Population	$O(N \times (C + P \times (R + A + B)))$
	Q-Table	$O(S \times K)$ (shared across population)
	Total	$O(N \times (C + P \times (R + A + B)) + S \times K)$

Table 4
Experimental test instances.

Instance	Size	Number of teachers	Number of cross campus teachers	Number of course tasks	Number of classes	Number of campuses	Weeks per semester
1	60_6_100_40_2	60	6	100	40	2	16
2	120_12_200_80_2	120	12	200	80	2	16
3	210_21_350_140_2	210	21	350	140	2	16
4	300_30_500_200_2	300	30	500	200	2	16
5	60_6_100_40_3	60	6	100	40	3	16
6	120_12_200_80_3	120	12	200	80	3	16
7	210_21_350_140_3	210	21	350	140	3	16
8	300_30_500_200_3	300	30	500	200	3	16
9	60_6_100_40_4	60	6	100	40	4	16
10	120_12_200_80_4	120	12	200	80	4	16
11	210_21_350_140_4	210	21	350	140	4	16
12	300_30_500_200_4	300	30	500	200	4	16
13	60_6_100_40_5	60	6	100	40	5	16
14	120_12_200_80_5	120	12	200	80	5	16
15	210_21_350_140_5	210	21	350	140	5	16
16	300_30_500_200_5	300	30	500	200	5	16
17	60_6_100_40_6	60	6	100	40	6	16
18	120_12_200_80_6	120	12	200	80	6	16
19	210_21_350_140_6	210	21	350	140	6	16
20	300_30_500_200_6	300	30	500	200	6	16

Table 5
Parameter levels.

Parameter	Factor level			
	1	2	3	4
PS	50	80	100	120
α	0.1	0.3	0.5	0.7
γ	0.3	0.5	0.7	0.9

optimal action based on the q-value. The calculation formula is shown in Eq. (23):

$$a = \begin{cases} \text{randomaction} & \text{rand}(0,1) < \varepsilon \\ \text{argmax}_{a'} Q(s,a') & \text{otherwise} \end{cases} \tag{23}$$

where a is the selected action; ε is the exploration rate that controls the probability of random action selection; and $\max_{a'}Q(s',a')$ is the Q-value, which estimates the long-term reward of taking action a in state s.

The exploration rate is adjusted based on hypervolume stagnation to avoid premature convergence, the calculation formula is shown in Eq. (24).

$$\varepsilon = \begin{cases} \min(0.8, 1.2\varepsilon) & \text{if} HV_t - HV_{t-10} < 0.001 \\ \max(0.05, 0.995\varepsilon) & \text{otherwise} \end{cases} \tag{24}$$

where HV_t *and* $HV_{t\text{-}10}$ are the hypervolume values at the current iteration t and iteration t-10, respectively; 0.001 is the threshold for judging whether hypervolume improvement is stagnant; and 0.8 and 0.05 are the upper and lower bounds for ε to avoid extreme values.

(4) **Reward Function:** Reward serves as the core feedback signal in Q-learning, guiding the agent to distinguish between good and bad actions, reinforcing optimal decisions (by increasing the corresponding Q-value) and weakening suboptimal choices, thereby enabling it to ultimately learn the optimal action strategy. Specifically, the designed reward function quantifies the progress of multi-objective optimization by measuring the improvement of each objective relative to a reference point, thus providing meaningful feedback for the agent to pursue optimal trade-offs among multiple goals, the calculation formula is shown in Eq. (25).

$$R = \sum_{i=1}^{m} \max\left(\frac{f_i^{\text{old}} - f_i^{\text{new}}}{r_i}, 0\right) \tag{25}$$

where R is the total reward; f old $i$and f new $i$are the i th objective values before and after the action, respectively; m is the number of objectives, and r_i is the i th coordinate of the reference point.

(5) **Update Mechanism**: This mechanism updates Q-values and adjusts the exploration rate to refine learning and adapt search strategies. Q-values are updated using temporal difference learning to integrate current and future rewards, the calculation formula is shown in Eq. (26).

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a')] \tag{26}$$

where α is the learning rate that weights new information; γ is the discount factor that weights future rewards; R is the immediate reward after action execution; s' is the next state after taking action a; and $\max_{a'}Q(s',a')$ is the maximum Q-value of all possible actions in state s', representing the best expected future reward.

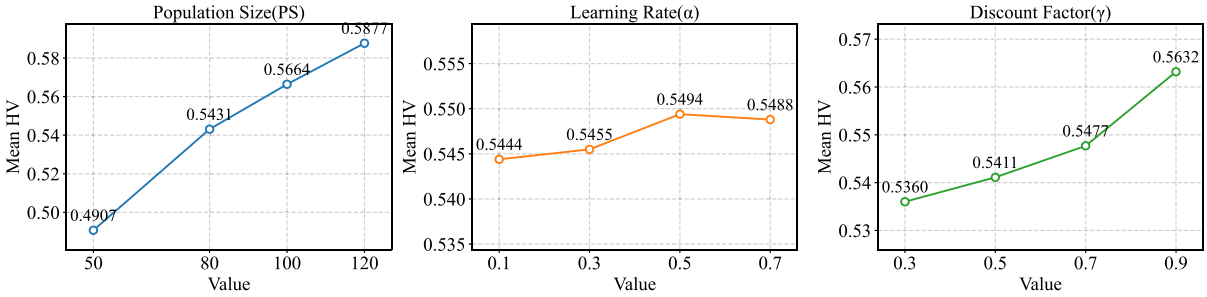
3.4. Low-level heuristic

To effectively explore the solution space of the MCUCSP and enhance solution quality, this paper designs and implements a systematic set of low-level heuristic (LLH) operators. These operators progressively optimize the objective function values through local perturbation or recombination of the current solution while ensuring that all hard constraints remain satisfied, embodying a search strategy of local improvement with global convergence. Specifically, the operator design adheres to the principles of constraint consistency, where all operators are verified by a hard-constraint checking mechanism before and after execution to guarantee feasibility; goal orientation, where greedy strategies are incorporated in selected operators to prioritize improvements in preference matching or conflict reduction; and diversity maintenance, where randomized operators introduce probabilistic perturbations to prevent premature convergence and enhance global exploration capability.

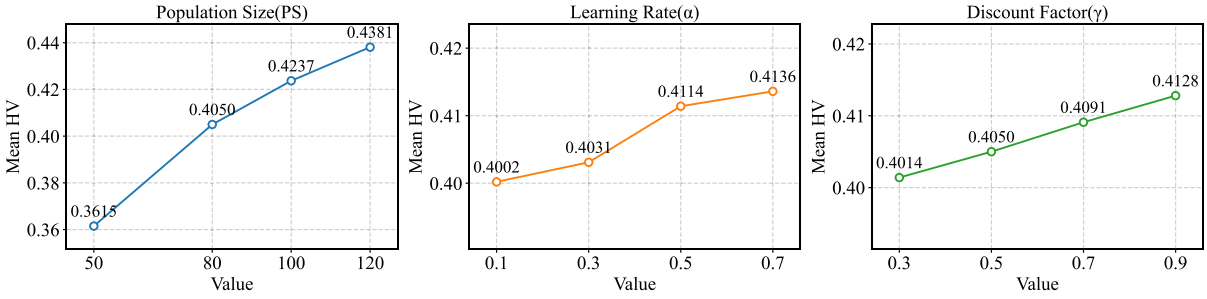
3.4.1. Heuristic operators

The low-level heuristics consist of ten different operation operators, divided into two categories: random operators and greedy operators. Random operators (LLH1–LLH4) introduce strong randomness, helping escape local optima and explore the solution space broadly. Greedy operators (LLH5–LLH10) are highly directional, optimizing specific objectives to achieve better solutions in targeted directions. The organic combination of these operators balances exploration and exploitation. The details of ten operators are as follows.

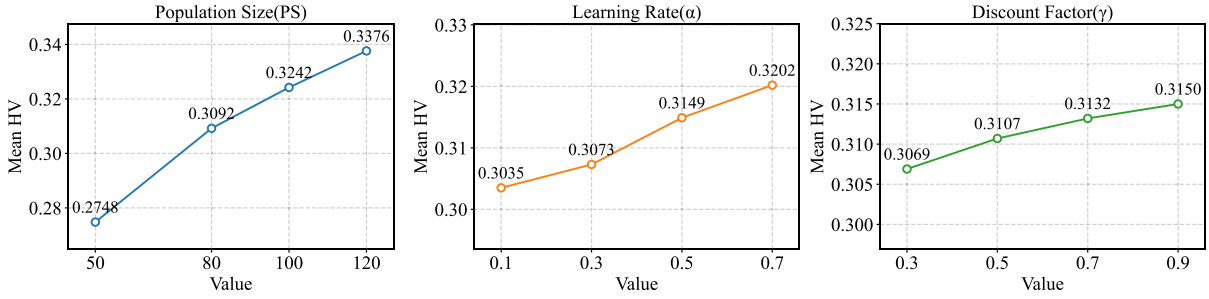
Group A (Random Operators, LLH1–LLH4), as shown in Fig. 3.



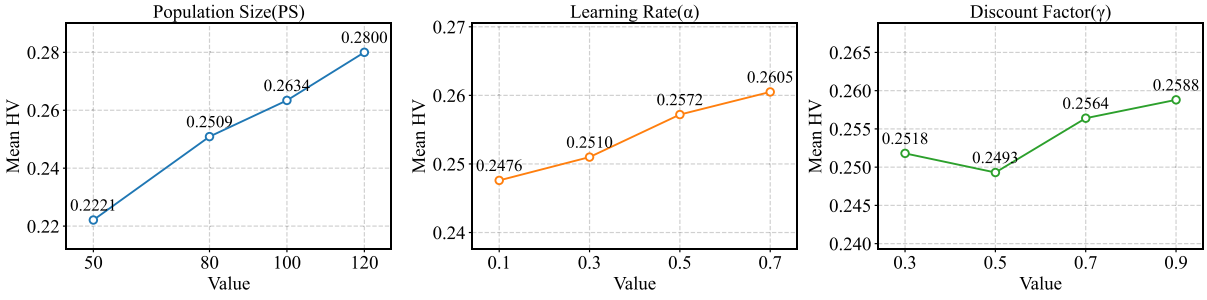
(a) Main Effect of Key Parameters for instance 1(60_6_100_40_2)



(b) Main Effect of Key Parameters for instance 2(120_12_200_80_2)



(c) Main Effect of Key Parameters for instance 3(210_21_350_140_2)



(d) Main Effect of Key Parameters for instance 4(300_30_500_200_2)

Fig. 7. Main effect plots of various parameters under different instance scales.

LLH1 (Course Swap Operator): Randomly selects two course tasks within the same campus and swaps their timeslots and classrooms, with bidirectional checks for classroom capacity and type compatibility.

LLH2 (Course Move Operator): Randomly relocates a single course to an available time-slot–classroom combination within the same campus, prioritizing already-utilized classrooms to improve resource utilization.

LLH4 (Timeslot Swap Operator): Swaps all course assignments between two distinct timeslots (including empty ones) within the same campus, supporting both cross-classroom and same-classroom scenarios.

LLH4 (Campus Swap Operator): Exchanges all course schedules and classroom resources of a given campus between two solutions, ensuring identical task sets and repairing cross-campus teacher conflicts via local adjustments.

Group B (Greedy Operators, LLH5–LLH10), as shown in Fig. 4.

LLH5 (Evening Course Adjustment Operator): Moves courses scheduled in evening slots (mod 5 = 4) to non-evening free slots or swaps them with larger courses to reduce evening teaching load.

LLH6 (Over-Scheduled Class Dispersion Operator): Detects repeated scheduling of the same class on the same day and disperses these courses across different weekdays via moves or swaps.

LLH7 (Teacher Preference Optimization Operator): Adjusts courses to timeslots with higher preference scores based on teacher-specific day and period preferences.

LLH8 (Course-Period Adaptation Operator: Moves courses to more suitable periods according to course-specific period preferences (e.g., preferring mornings for theoretical courses).

LLH9 (Cross-Campus Commute Reduction Operator): Identify teachers who teach across different campuses, and minimize their commuting time by arranging their teaching tasks at the same campus on the same day as much as possible.

LLH10 (Classroom Utilization Minimization Operator): Selects the least-utilized classroom and migrates its courses to higher-usage classrooms to reduce the total number of active classrooms.

3.4.2. General execution process of low-level heuristics

All LLH operators follow a unified execution framework to ensure modularity and extensibility. The process takes the encoded representation of the current solution as input and realizes local optimization through a closed loop of decode–verify–evaluate. Initially, the operator creates a copy of the current encoding and applies a local perturbation (swap, move, or recombination) to generate a new encoding. Subsequently, a decoder converts the encoding into a list of schedule objects, and a hard-constraint checking module verifies feasibility. If the new solution satisfies all constraints, an objective function calculator evaluates its quality. If the new solution outperforms the incumbent or meets an acceptance criterion, it replaces the current solution and updates the search state. This generate-test-improve cycle is repeated during algorithm iterations until termination criteria are met. Notably, LLH4 operator require additional handling of dual-solution interactions and classroom resource synchronization. Algorithm 2 demonstrates the specific execution process.

3.5. Framework of MO-QL-HH

The proposed MO-QL-HH algorithm employs a two-layer architecture that integrates reinforcement learning with evolutionary search. Initially, a greedy initialization method (Algorithm 1) creates a feasible population by sorting tasks according to difficulty scores and promoting classroom reuse, thus establishing the first Pareto front. In each generation, every solution is encoded as a tuple (task–campus–classroom–timeslot–week–order). A high-level Q-learning agent then discretizes the normalized objective vector into a state and chooses an action from ten low-level heuristics (LLH1–LLH10) via an ε-greedy policy (Algorithm 2). This strategy balances exploration (random selection of LLH1–4) and exploitation (greedy selection of LLH5–10). The selected heuristic applies a local perturbation to the schedule, after which the solution is decoded and repaired to satisfy all hard constraints. A reward, calculated from the improvement of each objective relative to a reference point, is used to update the Q-table, while the exploration rate ε is adjusted based on hypervolume stagnation. Offspring compete with parent individuals; unsuccessful mutants probabilistically replace the worst-performing individuals or re-sample elite individuals from the archive. The overall framework of the proposed MO-QL-HH is shown in Fig. 5, with a detailed flowchart in Fig. 6 and pseudocode in Algorithm 3.

3.6. Computational complexity analysis of MO-QL-HH

This section analyzes the time and space complexity of the MO-QL-HH algorithm. Assume the population size is N, the maximum number of iterations is T, the number of low-level heuristics (LLHs) is K ($K = 10$ in MO-QL-HH), the number of timeslots is P, the number of classrooms is R, the number of course tasks is C, the number of teachers is A, and the number of classes is B.

Table 6
ANOVA results of different instances.

Scale	Source	Sum Sq.	d. f.	Mean Sq.	*F*-ratio	*p*-value
100	*PS*	0.8347	3	0.2782	217.0763	**0.0000**
	α	0.0029	3	0.0010	0.7485	0.5235
	γ	0.0672	3	0.0224	17.4834	**0.0000**
	$PS^*\alpha$	0.0047	9	0.0005	0.4055	0.9323
	$PS^*\gamma$	0.0070	9	0.0008	0.6044	0.7938
	$\alpha^*\gamma$	0.0157	9	0.0017	1.3626	0.2018
	$PS^*\alpha^*\gamma$	0.0203	27	0.0008	0.5868	0.9536
	Residual	0.7383	576	0.0013		
200	*PS*	0.5307	3	0.1769	316.3139	**0.0000**
	α	0.0198	3	0.0066	11.8097	**0.0000**
	γ	0.0118	3	0.0039	7.0519	**0.0001**
	$PS^*\alpha$	0.0093	9	0.0010	1.8409	0.0584
	$PS^*\gamma$	0.0053	9	0.0006	1.0571	0.3931
	$\alpha^*\gamma$	0.0063	9	0.0007	1.2581	0.2569
	$PS^*\alpha^*\gamma$	0.0140	27	0.0005	0.9249	0.5760
	Residual	0.3221	576	0.0006		
350	*PS*	0.3504	3	0.1168	289.4312	**0.0000**
	α	0.0272	3	0.0091	22.4699	**0.0000**
	γ	0.0060	3	0.0020	4.9191	**0.0022**
	$PS^*\alpha$	0.0054	9	0.0006	1.5004	0.1442
	$PS^*\gamma$	0.0045	9	0.0005	1.2265	0.2757
	$\alpha^*\gamma$	0.0020	9	0.0002	0.5555	0.8336
	$PS^*\alpha^*\gamma$	0.0097	27	0.0004	0.8912	0.6259
	Residual	0.2325	576	0.0004		
500	*PS*	0.2862	3	0.0954	305.8955	**0.0000**
	α	0.0163	3	0.0054	17.4588	**0.0000**
	γ	0.0090	3	0.0030	9.5714	**0.0000**
	$PS^*\alpha$	0.0060	9	0.0007	2.1293	**0.0255**
	$PS^*\gamma$	0.0028	9	0.0003	1.0075	0.4326
	$\alpha^*\gamma$	0.0013	9	0.0001	0.4588	0.9020
	$PS^*\alpha^*\gamma$	0.0131	27	0.0005	1.5526	**0.0382**
	Residual	0.1796	576	0.0003		

* Bold denotes statistically significant results ($p < 0.05$).

Table 7
The suggested parameters for task instances of different scales.

Number of course task	*PS*	α	γ
100	120	0.7	0.9
200	120	0.5	0.7
350	120	0.7	0.7
500	120	0.7	0.3

Table 8
Parameter settings of all comparison algorithms.

Algorithm	Special parameters	General parameters
NSGA-II	-	Population size $N = 120$,
MOEA/D	Number of neighboring vectors $T = 10$	Number of generations $G = 100$,
NSGA-III	-	Crossover probability $Pc = 0.90$,
SPEA2	Archive size AS=100	
MOEA/D-AAWNs	Init neighbor size NS=10 First Adjustment of Neighborhoods α=0.1 Second Adjustment of Neighborhoods β=0.05	Mutation probability $Pm = 0.10$
HGTSA	Tabu tenure TT=15	

3.6.1. Time complexity analysis

(1) Population Initialization: Generating the initial population of size N requires, for each individual, sorting all C course tasks by a heuristic priority such as class size or weekly hours, which costs $O(C \times \log C)$. After sorting, every task must be assigned to a feasible classroom–timeslot pair among P periods and R classrooms while satisfying hard constraints (room type, capacity, teacher unavailability, etc.). Scanning the two-dimensional $P \times R$ space for

Table 9
The HV results of MO-QL-HH and all comparative algorithms.

Instance	MO-QL-HH		NSGA2		MOEA/D		NSGA3		SPEA2		MOEA/D-AAWNs		HGTSA	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
1	**0.6262**	0.0458	0.4329	0.0209	0.3916	0.0651	0.4073	0.0336	0.4355	**0.0188**	0.3766	0.0518	0.4249	0.0468
2	**0.4566**	0.0272	0.3361	**0.0200**	0.2931	0.0438	0.2997	0.0230	0.3079	0.0223	0.2816	0.0440	0.2787	0.0639
3	**0.3771**	0.0176	0.2589	0.0206	0.2106	0.0225	0.2444	0.0215	0.2500	**0.0164**	0.2216	0.0382	0.1997	0.0164
4	**0.2736**	0.0197	0.1860	0.0109	0.1507	0.0329	0.1838	0.0121	0.1766	**0.0094**	0.1457	0.0215	0.1376	0.0130
5	**0.5837**	0.0253	0.4825	0.0275	0.4285	0.0828	0.4583	**0.0244**	0.4901	0.0320	0.3974	0.0466	0.4343	0.0571
6	**0.5933**	0.0386	0.4684	0.0373	0.3875	0.0602	0.4233	**0.0265**	0.4358	0.0291	0.4074	0.0420	0.3642	0.0362
7	**0.3076**	0.0261	0.2338	0.0161	0.1844	0.0141	0.2158	0.0123	0.2230	**0.0118**	0.1803	0.0243	0.1690	0.0162
8	**0.2439**	0.0150	0.1855	0.0115	0.1525	0.0226	0.1709	**0.0056**	0.1834	0.0073	0.1419	0.0284	0.1544	0.0166
9	**0.7081**	0.0361	0.6170	**0.0169**	0.5431	0.0541	0.5822	0.0219	0.6339	0.0377	0.5780	0.0793	0.5225	0.1067
10	**0.6014**	0.0342	0.5276	0.0518	0.3986	0.0471	0.5039	0.0240	0.4878	**0.0234**	0.4032	0.0359	0.4363	0.0491
11	**0.3266**	0.0146	0.2704	**0.0107**	0.2608	0.0430	0.2541	0.0138	0.2600	0.0133	0.2207	0.0287	0.2211	0.0221
12	**0.2311**	0.0118	0.1809	0.0106	0.1502	0.0281	0.1751	0.0101	0.1727	**0.0077**	0.1351	0.0118	0.1856	0.0057
13	**0.6649**	0.0530	0.6111	0.0275	0.5007	0.0569	0.5638	0.0264	0.6035	0.0323	0.5375	0.0309	0.6151	**0.0254**
14	**0.4966**	0.0305	0.4544	0.0331	0.3595	0.0564	0.4309	0.0220	0.4294	0.0270	0.3994	0.0603	0.4892	**0.0213**
15	**0.4121**	0.0217	0.3436	0.0227	0.2964	0.0631	0.3462	**0.0165**	0.3319	0.0226	0.2643	0.0297	0.3718	0.0317
16	**0.2376**	0.0154	0.2160	**0.0090**	0.1703	0.0266	0.2126	0.0166	0.2050	0.0112	0.1535	0.0134	0.2220	0.0108
17	0.4879	0.0200	0.4704	0.0212	0.3972	0.0440	0.4593	**0.0141**	0.4774	0.0169	0.3996	0.0418	**0.4992**	0.0181
18	**0.4365**	0.0222	0.4037	0.0183	0.3198	0.0387	0.3835	0.0164	0.3881	0.0228	0.3383	0.0284	0.4173	**0.0120**
19	**0.3056**	0.0232	0.2716	0.0176	0.2038	0.0212	0.2539	0.0163	0.2486	**0.0141**	0.2084	0.0176	0.2619	0.0219
20	**0.2202**	0.0132	0.2062	**0.0072**	0.1569	0.0119	0.1944	0.0099	0.2024	0.0120	0.1573	0.0104	0.2157	0.0140

Table 10
The IGD results of MO-QL-HH and all comparative algorithms.

Instance	MO-QL-HH		NSGA2		MOEA/D		NSGA3		SPEA2		MOEA/D-AAWNs		HGTSA	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
1	**0.0050**	**0.0121**	0.2913	0.0776	0.3404	0.1205	0.3622	0.0936	0.3079	0.0851	0.3826	0.1167	0.2884	0.0730
2	**0.0101**	**0.0076**	0.1332	0.0590	0.1632	0.0517	0.1703	0.0572	0.1659	0.0548	0.2104	0.0486	0.2129	0.0996
3	**0.0111**	**0.0121**	0.1278	0.0351	0.1653	0.0248	0.1401	0.0345	0.1289	0.0284	0.1747	0.0316	0.1765	0.0314
4	**0.0139**	**0.0184**	0.1200	0.0189	0.1885	0.1016	0.1116	0.0186	0.1244	0.0224	0.1573	0.0271	0.1664	0.0368
5	**0.0681**	0.0868	0.1952	**0.0560**	0.2810	0.1111	0.2404	0.0602	0.1826	0.0796	0.3061	0.0973	0.2771	0.0843
6	**0.0096**	**0.0095**	0.1218	0.0649	0.2076	0.0829	0.1840	0.0737	0.1517	0.0424	0.1560	0.0426	0.2028	0.0341
7	**0.0168**	**0.0166**	0.0982	0.0255	0.1497	0.0343	0.1117	0.0276	0.1236	0.0406	0.1711	0.0454	0.1689	0.0493
8	**0.0240**	0.0317	0.1001	0.0322	0.1496	0.0484	0.1325	**0.0290**	0.1176	0.0376	0.1744	0.0482	0.1510	0.0315
9	**0.0999**	0.0806	0.2055	0.0563	0.2877	0.0954	0.2583	0.0890	0.1765	0.0913	0.2631	**0.0475**	0.3302	0.0911
10	**0.0442**	0.0356	0.1134	0.0503	0.1911	0.0429	0.1220	**0.0333**	0.1449	0.0587	0.1798	0.0845	0.2066	0.0996
11	**0.0239**	0.0278	0.0790	0.0224	0.1820	0.1379	0.0939	0.0222	0.0902	**0.0176**	0.1470	0.0570	0.1343	0.0303
12	**0.0600**	0.0260	0.0940	**0.0133**	0.1772	0.0647	0.1088	0.0209	0.1184	0.0213	0.1650	0.0256	0.1013	0.0216
13	**0.0399**	0.0415	0.1073	0.0514	0.1864	0.0550	0.1778	0.0666	0.1054	0.0429	0.1623	**0.0339**	0.1152	0.0478
14	**0.0740**	0.0586	0.1153	**0.0346**	0.2431	0.0947	0.1417	0.0555	0.1299	0.0356	0.2323	0.1537	0.0864	0.0398
15	**0.0291**	0.0213	0.0856	0.0345	0.2316	0.1302	0.0873	0.0254	0.0873	0.0299	0.1510	0.0510	0.0547	**0.0164**
16	0.0599	0.0248	0.0690	**0.0209**	0.1448	0.0818	0.0871	0.0266	0.0777	0.0221	0.1463	0.0363	**0.0565**	0.0286
17	0.0513	**0.0132**	0.0976	0.0523	0.1923	0.0652	0.1219	0.0441	0.0856	0.0430	0.1816	0.0809	**0.0486**	0.0305
18	**0.0257**	**0.0128**	0.0693	0.0265	0.1445	0.0498	0.0871	0.0349	0.0835	0.0342	0.1296	0.0512	0.0614	0.0334
19	**0.0225**	0.0192	0.0708	0.0228	0.1459	0.0409	0.0940	**0.0136**	0.0944	0.0325	0.1505	0.0557	0.0746	0.0144
20	0.0772	0.0252	0.0621	**0.0153**	0.1250	0.0239	0.0879	0.0156	0.0815	0.0190	0.1221	**0.0153**	**0.0535**	0.0253

every task gives $O(P \times R \times C)$. Hence, the combined cost for one solution is $O(C \times \log C + P \times R \times C)$ and for the whole population $O(N \times C \times (\log C + P \times R))$.

(2) Objective Function Evaluation: Each generation evaluates all N individuals against multiple objectives (teacher commute, room usage, night hours, over-assigned hours, teacher preference, course-period suitability). For a single individual, each of the C scheduled tasks is inspected together with its associated period, room, teacher, and classes. Aggregating these values over P periods, A teachers and B classes yields $O(C \times (P + A + B))$ per individual. Across the population this becomes $O(N \times C \times (P + A + B))$.

(3) Q-Learning Process: In every generation, each of the N individuals undergoes one reinforcement-learning decision step: Action selection via ε-greedy over K low-level heuristics costs $O(K)$. Reward computation and Q-table update are constant-time $O(1)$. Thus the per-generation overhead is $O(N \times K)$.

(4) Low-Level Heuristics: The time complexity varies by heuristic type when applied to N individuals. LLH1 (Course Swap) is $O(R)$; LLH2 (Course Move) and LLH10 (Reduce Classroom Usage) are $O(P \times R)$; LLH3–LLH9 are dominated by $O(P^2 \times R)$ due to nested scans over timeslots and rooms. The overall complexity for low-level heuristics is $O(N \times P^2 \times R)$.

Combined Per-Generation Cost: Summing the above components (objective function evaluation, Q-learning, and low-level heuristics) gives a per-generation complexity of $O(N \times (K + C \times (P + A + B) + P^2 \times R))$. Total Time Complexity: Including the one-time initialization cost and extending the per-generation complexity over T generations, the overall time complexity of the MO-QL-HH algorithm is as shown in Eq. (27).

$$O(\mathrm{MO-QL-HH}) = O\big(N \times C \times (\log C + P \times R) + T \times N \times \big(K + C \times (P + A + B) + P^2 \times R\big)\big) \tag{27}$$

3.6.2. Space complexity analysis

The algorithm allocates storage for two primary components: population-related data structures and the shared Q-learning table. For each of the N individuals in the population, storage is required for: (1)

Table 11
The C-metric results of MO-QL-HH and all comparative algorithms.

Instance	A:MO-QL-HH, A1:NSGA-II, A2:MOEA/D, A3:NSGA-III, A4:SPEA2, A5:MOEA/D-AAWNs, A6:HGTSA											
	C(A,A1)	C(A1,A)	C(A,A2)	C(A2,A)	C(A,A3)	C(A3,A)	C(A,A4)	C(A4,A)	C(A,A5)	C(A1,A5)	C(A,A6)	C(A6,A)
1	**0.9769**	0.0043	**0.8000**	0.0512	**0.9000**	0.0094	**0.9917**	0.0063	**0.9000**	0.0031	**0.9000**	0.0469
2	**0.7739**	0.0244	**0.7587**	0.0429	**0.9383**	0.0146	**0.9609**	0.0000	**0.6885**	0.0271	**0.8167**	0.0779
3	**0.8639**	0.0193	**0.9000**	0.0124	**0.8502**	0.0115	**0.9272**	0.0022	**0.7771**	0.0400	**0.9875**	0.0000
4	**0.8396**	0.0142	**0.7919**	0.0335	**0.8187**	0.0323	**0.9135**	0.0111	**0.8200**	0.0379	**0.9667**	0.0000
5	**0.8737**	0.0078	**0.7000**	0.1616	**0.8551**	0.0000	**0.7363**	0.0188	**0.9000**	0.0135	**0.8000**	0.0000
6	**0.8372**	0.0277	**0.7571**	0.1270	**0.9517**	0.0000	**0.9522**	0.0000	**0.7698**	0.0482	**1.0000**	0.0000
7	**0.7286**	0.0639	**0.7142**	0.0142	**0.8848**	0.0020	**0.8494**	0.0103	**0.7793**	0.0144	**0.9267**	0.0061
8	**0.8785**	0.0083	**0.8289**	0.0322	**0.8975**	0.0113	**0.8077**	0.0269	**0.8615**	0.0010	**0.8337**	0.0052
9	**0.7822**	0.0033	**0.7400**	0.0542	**0.8250**	0.0000	**0.6117**	0.0048	**0.5917**	0.1294	**0.8250**	0.1095
10	**0.5284**	0.2229	**0.9333**	0.0074	**0.7388**	0.0542	**0.8342**	0.0567	**0.9933**	0.0000	**0.7976**	0.0299
11	**0.7090**	0.0467	**0.5410**	0.1102	**0.8367**	0.0046	**0.8468**	0.0086	**0.8107**	0.0000	**0.9552**	0.0034
12	**0.6722**	0.0097	**0.5758**	0.0053	**0.6768**	0.0065	**0.7832**	0.0096	**0.9467**	0.0000	**0.4479**	0.0192
13	**0.6280**	0.0982	**0.8444**	0.0222	**0.8204**	0.0053	**0.7472**	0.0601	**0.5667**	0.0177	**0.6242**	0.1268
14	**0.6216**	0.0961	**0.7773**	0.0419	**0.6803**	0.3910	**0.6757**	0.1092	**0.5488**	0.1457	**0.4000**	0.3257
15	**0.7718**	0.0856	**0.5273**	0.0641	**0.7498**	0.1050	**0.8796**	0.0173	**0.8203**	0.0215	**0.5821**	0.1766
16	**0.6242**	0.0569	**0.7744**	0.0000	**0.5233**	0.0837	**0.6668**	0.0655	**0.8681**	0.0012	**0.4055**	0.0359
17	**0.5809**	0.1078	**0.8214**	0.0577	**0.6525**	0.0543	**0.4879**	0.2140	**0.6400**	0.1189	0.3054	**0.3641**
18	**0.6834**	0.0706	**0.6339**	0.0315	**0.7978**	0.0369	**0.7010**	0.1210	**0.6278**	0.0733	**0.3739**	0.3403
19	**0.5615**	0.2319	**0.6224**	0.0749	**0.6715**	0.0970	**0.7549**	0.0996	**0.6522**	0.0546	**0.6573**	0.1573
20	**0.4830**	0.0424	**0.7093**	0.0000	**0.5278**	0.0666	**0.5556**	0.0189	**0.5792**	0.0000	**0.3555**	0.0883

Table 12
The CPU runtime(s) of MO-QL-HH and all comparative algorithms.

Instance	MO-QL-HH	NSGA-II	MOEA/D	NSGA-III	SPEA2	MOEA/D-AAWNs	HGTSA
1	88.75	**54.79**	66.39	95.38	105.97	64.91	62.69
2	240.77	**152.28**	195.67	285.71	335.27	176.76	185.84
3	615.31	**467.88**	483.93	846.79	903.40	480.51	489.17
4	1149.58	**880.20**	924.92	1582.68	1755.69	897.04	895.00
5	62.17	**46.29**	57.22	76.38	83.64	54.88	56.35
6	220.95	**148.41**	176.04	256.82	273.44	156.73	161.75
7	503.77	**376.73**	446.53	702.36	730.96	435.75	446.01
8	1056.52	**813.21**	865.27	1451.80	1450.82	907.60	844.15
9	57.74	**37.28**	57.79	61.33	68.77	52.67	52.00
10	163.20	**133.12**	173.96	255.40	301.06	173.43	168.53
11	393.23	**369.93**	453.77	676.55	633.00	458.65	447.88
12	763.12	675.14	804.29	1114.94	1155.93	791.77	**664.38**
13	50.48	**43.74**	65.32	68.43	80.58	61.55	45.92
14	116.22	**107.49**	154.17	186.19	191.31	145.09	112.08
15	341.99	**267.86**	374.91	495.79	505.44	365.43	276.44
16	**620.98**	705.14	836.61	1177.16	1236.02	819.91	707.72
17	43.36	**41.17**	61.30	63.24	79.67	56.14	41.42
18	136.01	**101.75**	150.76	164.75	182.38	143.24	104.46
19	532.36	404.49	495.71	738.75	830.38	485.36	**404.27**
20	**785.71**	875.99	1084.38	1600.82	1704.48	1021.81	960.84
Mean time	397.11	**335.14**	396.45	595.06	630.41	387.46	356.34

the encoded schedule of C course-task tuples, which occupies $O(C)$ space; (2) the $P \times R$ classroom-timeslot allocation matrix, accounting for $O(P \times R)$ space; (3) the $P \times A$ teacher-timeslot matrix, with a space cost of $O(P \times A)$; and (4) the $P \times B$ class-timeslot matrix, requiring $O(P \times B)$ space. Integrating these components, the total storage for a single individual is $O(C + P \times R + P \times A + P \times B)$, which can be simplified to $O(C + P \times (R + A + B))$. Extending this to the entire population of size N, the cumulative space cost becomes $O(N \times (C + P \times (R + A + B)))$. Additionally, the shared Q-table (a core component of the reinforcement learning mechanism) stores a Q-value for each discrete state-action pair. With S normalized state buckets and K low-level heuristics (where $K = 10$ in MO-QL-HH), the Q-table requires $O(S \times K)$ space.

Combining the storage costs of the population-related data structures and the Q-table, the total space complexity of the MO-QL-HH algorithm is expressed as Eq. (28).

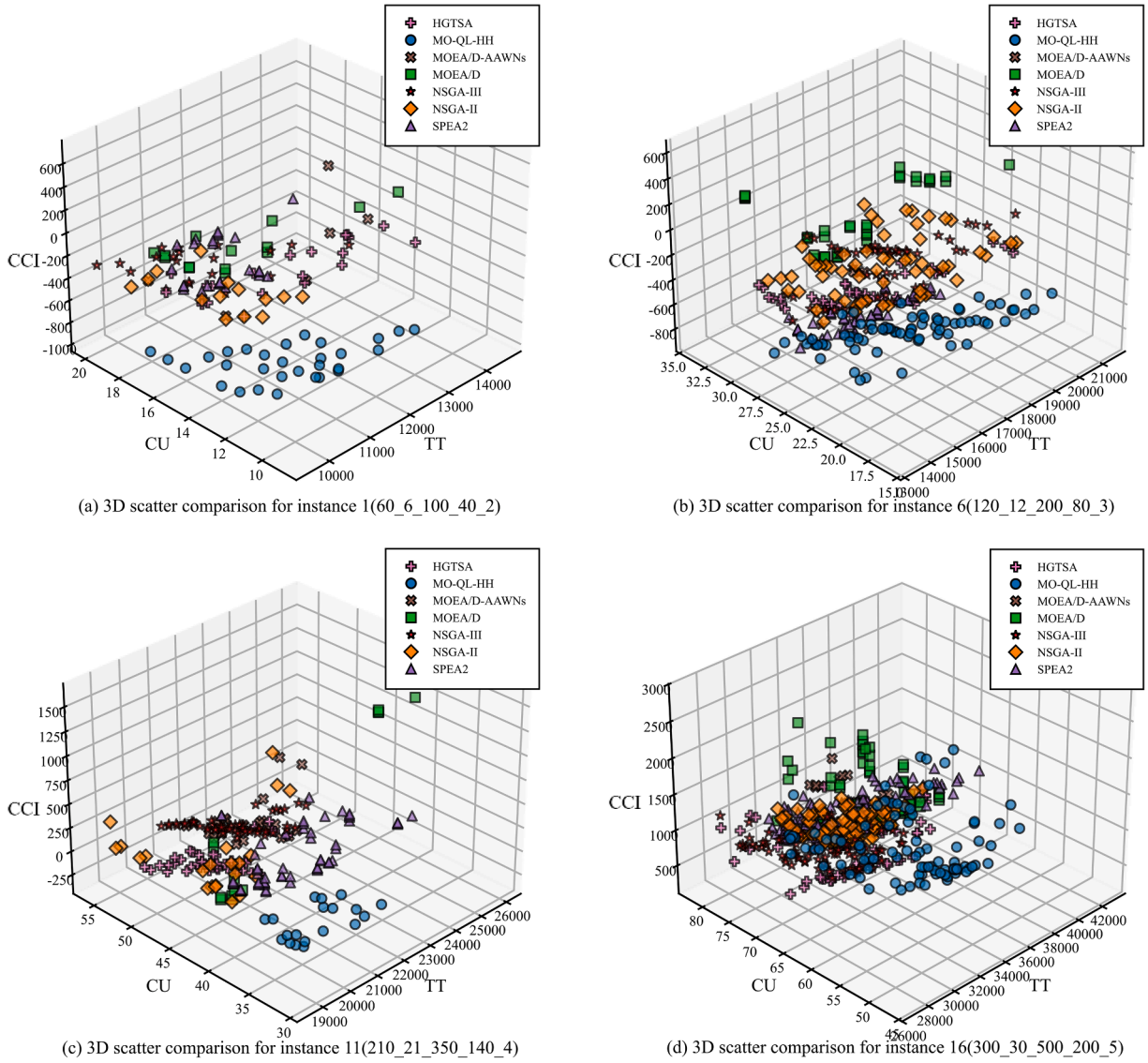
$$O(\mathrm{MO-QL-HH}) = O(N \times (C + P \times (R + A + B)) + S \times K) \tag{28}$$

A summary of the time complexity and space complexity is presented in Table 3.

4. Experimental results and analysis

4.1. Experimental setup

In this section, different scale instances are used to verify the performance of the model and algorithm. Since there are currently no widely recognized MCUCSP benchmark instances that simultaneously consider multi-campus and multi-dimensional soft constraint optimization, we have constructed test instances of different scales, we have constructed 20 instances based on the course offering plan of the College of Artificial Intelligence, Guangxi Minzu University to test the effectiveness of MO-QL-HH in optimizing multi-objective cross-campus UCSP. These 20 instances cover complex scenarios involving multiple scales of course tasks under different numbers of campuses (ranging from 2 to 6). The detailed settings of the instance data are presented in Table 4, and more specific details of the test instance data can be accessed via the https://github.com/zzzharden/MC_UCSP_data. The computational experiments in this study were conducted on a personal computer equipped with a 13th Generation Intel(R) Core(TM) i7-



(a) 3D scatter comparison for instance 1(60_6_100_40_2)

(b) 3D scatter comparison for instance 6(120_12_200_80_3)

(c) 3D scatter comparison for instance 11(210_21_350_140_4)

(d) 3D scatter comparison for instance 16(300_30_500_200_5)

Fig. 8. Comparison plots of 3D Pareto fronts of all algorithms on different instances.

13700HX CPU (2.10 GHz) and 16.0 GB of memory, with the programming implemented using the Python.

4.2. Performance metrics

(1) Hypervolume (HV): HV quantifies the m-dimensional Lebesgue measure of the region dominated by a non-dominated set S and bounded by a reference point r; larger values indicate better convergence and diversity. Let $S \subset \mathbb{R}^m$ denote the obtained non-dominated solution set and $r=(r_1,\ldots,r_m)$ be a reference point such that $s_i \le r_i$ for all $s \in S$ and every objective i. The calculation formula is given in Eq. (29).

$$HV(S,r) = \lambda\left(\bigcup_{s\in S}[s_1,r_1]\times\cdots\times[s_m,r_m]\right) \tag{29}$$

where $\lambda(\cdot)$ denotes the Lebesgue measure, s_i is the i th objective value of solution s, m is the number of objectives, and r_i is the i th coordinate of the reference point.

(2) Inverted Generational Distance (IGD): IGD measures the average Euclidean distance from uniformly sampled points on the true Pareto front to the nearest obtained solution; smaller values imply a better approximation. Let P^* be a set of reference points sampled from the true Pareto front and S be the obtained non-dominated set. The calculation formula is given in Eq. (30).

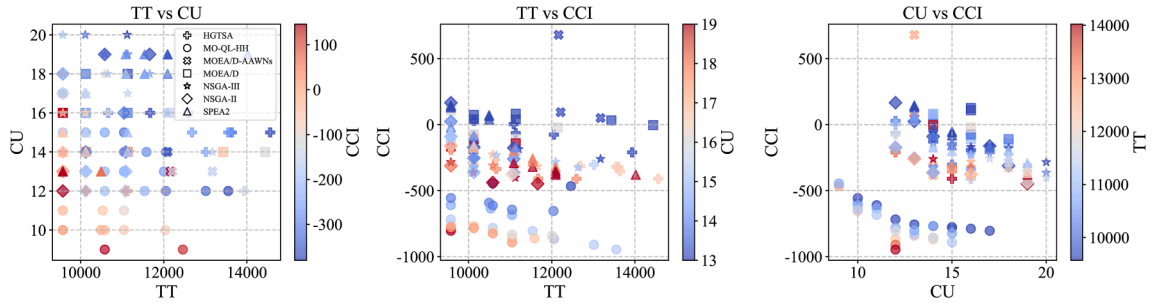
$$IGD(S,P^*) = \frac{1}{|P^*|}\sum_{p\in P^*}\min_{s\in S} d(x,y) \tag{30}$$

where p is a reference point in P^*, s is a solution in S, and $d(x, y)$ indicates the Euclidean distance calculated between the points x and y. $|P^*|$ is the cardinality of P^*, Moreover, P^* is the set of non-dominated solutions among the final solutions obtained by all algorithms in all runs. Before calculating the IGD, the non-dominated solutions of all algorithms must be normalized with respect to P^*. The calculation formula is given in Eq. (31).

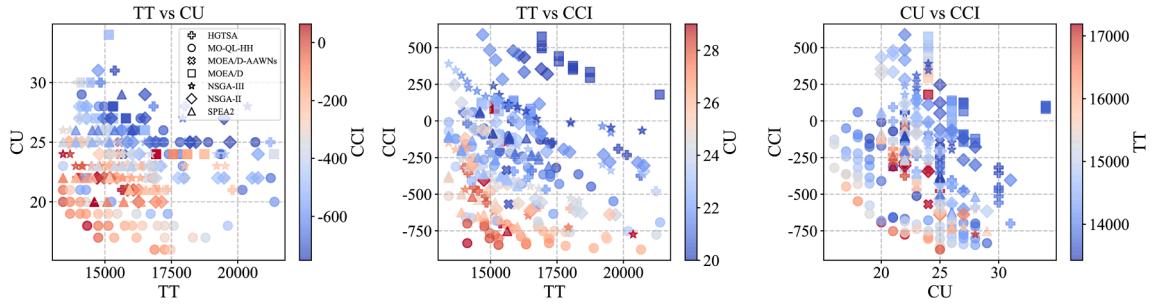
$$\bar{f}_i(x) = \frac{f_i(x) - f_i^{\min}}{f_i^{\max} - f_i^{\min}}, \quad i = 1,2,3 \tag{31}$$

where f max i and f min i are the maximal and minimal values of the i th objective function, respectively, among all solutions.

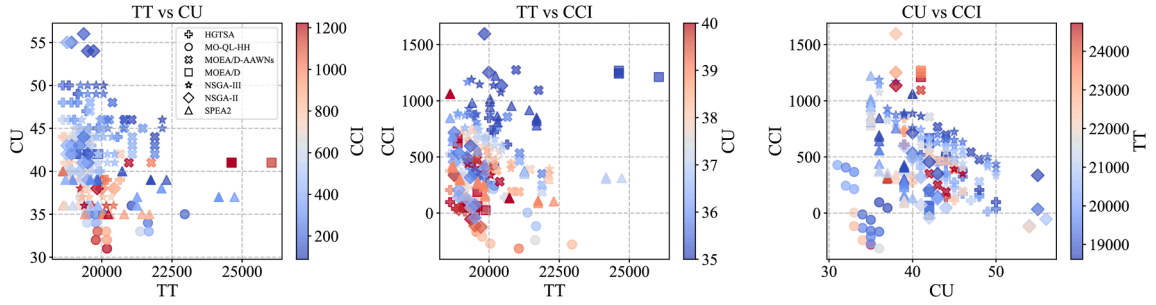
(3) Coverage Metric (C-metric): C-metric evaluates the dominance relationship between two non-dominated sets A and B, yielding a scalar in [0,1]; a value close to one indicates that A strongly dominates B. Let A and B be two non-dominated sets. The calculation formula is given in Eq.



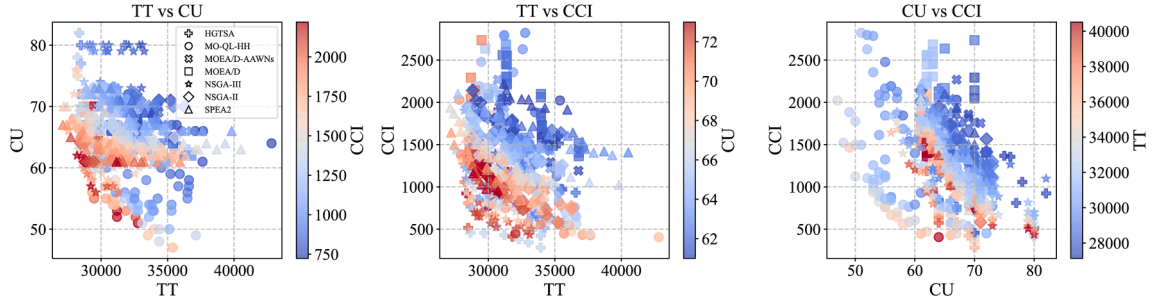
(a) 2D scatter comparison for instance 1(60_6_100_40_2)



(b) 2D scatter comparison for instance 6(120_12_200_80_3)



(c) 2D scatter comparison for instance 11(210_21_350_140_4)



(d) 2D scatter comparison for instance 16(300_30_500_200_5)

Fig. 9. Comparison plots of 2D Pareto fronts of all algorithms on different instances.

(32).

$$C(A,B) = \frac{|\{b \in B | \exists a \in A, a > b\}|}{|B|} \tag{32}$$

where $a > b$ signifies that solution a dominates solution b, and $|B|$ on behalf of the number of the solutions of B.

4.3. Parameter settings

Reasonable parameter settings play a crucial role in algorithm performance. To systematically analyze the impact of key parameters on the performance of MO-QL-HH, the study performs a parameter sensitivity analysis on three core parameters (learning rate α, discount factor γ, population size PS) based on the full factorial design in [54]. The full factorial design comprehensively investigates the main effects of each parameter and their interactions, avoiding suboptimal configurations that might arise from neglecting parameter couplings, thereby providing more reliable and robust parameter recommendations for the algorithm across problems of different scales.

Based on preliminary experiments and problem characteristics, four levels are set for each parameter (Table 5). For each task scale (100, 200, 350, and 500 course tasks), we execute all 4^3=64 parameter combinations on the corresponding test instances. Each combination is independently run 10 times, using the HV as the response variable (RV).

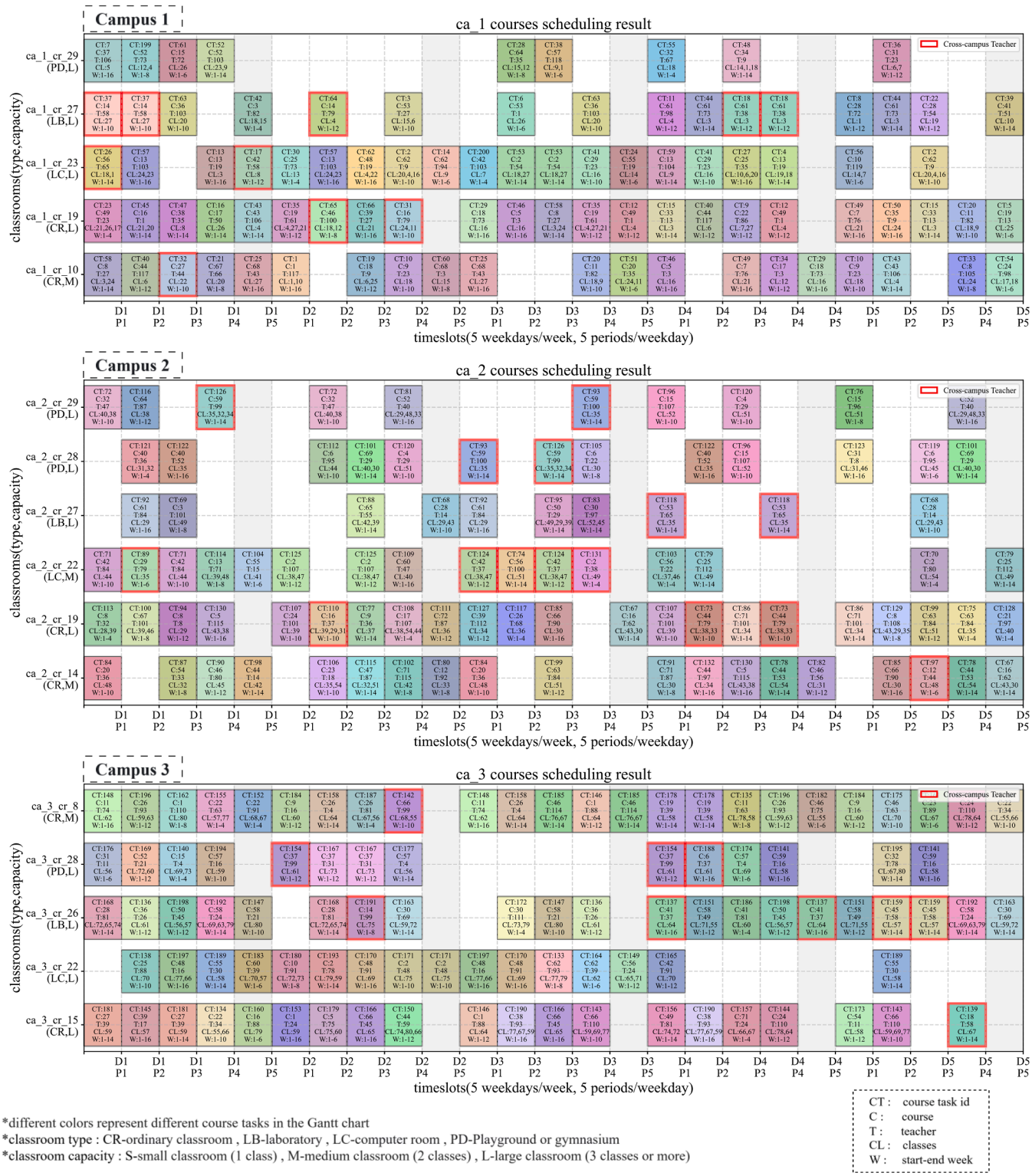


Fig. 10. The scheduling results of instance 5(120_12_200_80_3).

Analysis of variance (ANOVA) is used to assess the significant effects of each parameter and their interaction terms on the algorithm's performance. Fig. 7 illustrates the main effect trends of each parameter level on the HV value across different task scales. The ANOVA results in Table 6 indicate that the population size *PS* has a highly significant impact (p-value < 0.001) on the algorithm's performance across all task scales, establishing it as the most influential factor. The learning rate α and discount factor γ also show significance in most cases. Furthermore, for the large-scale instance with 500 courses, interaction effects between parameters reach significant levels. This underscores the necessity of examining synergistic effects between parameters within the full factorial framework, which might be obscured in partial factorial designs like the Taguchi method.

Based on the analysis of the mean RV from the full factorial experiments, we recommend the optimal parameter combinations for instances of different task scales, as shown in Table 7. These

Table 13
The HV and IGD results of MO-QL-HH and two variant algorithms.

Instance	HV						IGD					
	A1		A2		A3		A1		A2		A3	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std	Mean	Std
1	**0.5983**	**0.0215**	0.5313	0.0308	0.5084	0.0238	**0.0484**	**0.0259**	0.0903	0.0488	0.1089	0.0458
2	**0.4471**	**0.0184**	0.3982	0.0198	0.3278	0.0202	**0.0613**	0.0414	0.1421	0.0817	0.1307	**0.0263**
3	**0.3555**	0.0258	0.2987	0.0198	0.2261	**0.0115**	**0.0582**	0.0327	0.1076	0.0358	0.1580	**0.0284**
4	**0.2817**	0.0189	0.2262	0.0113	0.1681	**0.0098**	**0.0392**	0.0251	0.1089	0.0280	0.1714	**0.0192**
5	**0.6528**	0.0335	0.5874	**0.0173**	0.5604	0.0482	**0.0479**	0.0407	0.1357	0.0756	0.1326	**0.0398**
6	**0.5646**	0.0425	0.4859	0.0356	0.3708	**0.0201**	**0.0339**	**0.0285**	0.1339	0.0662	0.1801	0.0396
7	**0.3076**	0.0151	0.2686	0.0284	0.1954	**0.0096**	**0.0641**	**0.0241**	0.1283	0.0834	0.1631	0.0302
8	**0.1926**	0.0146	0.1695	0.0111	0.1186	**0.0073**	**0.0462**	**0.0170**	0.0666	0.0178	0.1448	0.0237
9	**0.7112**	0.0439	0.6993	0.0463	0.5573	**0.0120**	**0.0295**	**0.0250**	0.0921	0.0276	0.2043	0.0544
10	**0.6131**	0.0626	0.5599	**0.0317**	0.4290	0.0356	**0.0759**	0.0990	0.0895	**0.0434**	0.1826	0.0719
11	**0.3003**	**0.0115**	0.2049	0.0204	0.2805	0.0146	**0.0574**	**0.0266**	0.1136	0.0466	0.1418	0.0428
12	**0.2161**	0.0101	0.1862	0.0129	0.1301	**0.0067**	**0.0493**	**0.0188**	0.0754	0.0260	0.1525	0.0221
13	**0.6508**	**0.0235**	0.5920	0.0350	0.4848	0.0376	**0.0469**	0.0670	0.1196	**0.0518**	0.1771	0.0663
14	**0.5065**	0.0351	0.4644	**0.0222**	0.3603	0.0241	**0.0318**	**0.0364**	0.1222	0.0741	0.1771	0.0704
15	**0.4001**	0.0282	0.3880	0.0314	0.2594	**0.0205**	**0.0586**	**0.0444**	0.0939	0.0581	0.1944	0.0574
16	**0.2197**	0.0121	0.2082	0.0135	0.1302	**0.0081**	**0.0558**	**0.0262**	0.0643	0.0357	0.1571	0.0339
17	**0.6225**	0.0485	0.5730	0.0343	0.4873	**0.0334**	**0.0366**	**0.0379**	0.1819	0.1262	0.2090	0.1025
18	**0.4757**	0.0284	0.4414	0.0259	0.3548	**0.0242**	**0.0317**	**0.0212**	0.1005	0.0801	0.1455	0.0404
19	**0.2748**	0.0201	0.2605	0.0151	0.1809	**0.0139**	**0.0383**	**0.0259**	0.0870	0.0298	0.1416	**0.0259**
20	**0.2130**	0.0099	0.2052	**0.0083**	0.1229	0.0108	**0.0478**	0.0267	0.0565	**0.0184**	0.1557	0.0216

Table 14
The C-metric and CPU runtime results of MO-QL-HH and two variant algorithms.

Instance	C-metric					Time(s)		
	(A1, A2)	(A2, A1)	(A1, A3)	(A3, A1)		A1	A2	A3
1	**0.5916**	0.1617	**0.5469**	0.1268		155.11	**150.57**	176.30
2	**0.4727**	0.1220	**0.8051**	0.0000		**239.87**	246.93	329.23
3	**0.3040**	0.1049	**0.6087**	0.0000		**357.74**	483.32	378.91
4	**0.3433**	0.0922	**0.6959**	0.0100		494.54	796.56	**388.59**
5	**0.6706**	0.1826	**0.7580**	0.1197		68.31	73.29	**65.82**
6	**0.3877**	0.1695	**0.8750**	0.0069		**165.98**	180.45	204.64
7	**0.3317**	0.1611	**0.6960**	0.0000		309.97	441.86	**270.00**
8	**0.2856**	0.2073	**0.7979**	0.0040		**719.75**	773.54	1200.03
9	**0.5363**	0.1026	**0.9588**	0.0000		60.44	63.64	**60.21**
10	**0.4307**	0.2305	**0.8659**	0.0077		116.04	167.25	**110.07**
11	0.1177	**0.2849**	**0.7215**	0.0010		**241.35**	347.93	329.56
12	**0.1910**	0.1148	**0.7609**	0.0000		**459.66**	550.03	539.60
13	**0.6609**	0.0734	**0.9006**	0.0120		57.90	68.17	**51.33**
14	**0.5078**	0.2663	**0.9218**	0.0000		**97.75**	126.02	100.95
15	**0.3381**	0.3180	**0.8839**	0.0034		235.53	285.24	**221.40**
16	**0.2395**	0.1850	**0.8224**	0.0000		**385.57**	653.21	642.33
17	**0.5893**	0.1235	**1.0000**	0.0000		57.71	68.38	**51.94**
18	**0.3562**	0.2916	**0.8454**	0.0000		**104.38**	126.54	114.43
19	0.2766	**0.4362**	**0.9029**	0.0000		**356.90**	456.84	561.13
20	0.1232	**0.1461**	**0.9681**	0.0000		516.97	1318.97	**371.25**
Win numbers (Rate)	**17(85 %)**	3(15 %)	**20(100 %)**	0(0 %)	Mean time	**260.07**	368.94	308.39

recommendations aim to strike the best balance between algorithm performance and computational cost, providing effective guidance for practical applications.

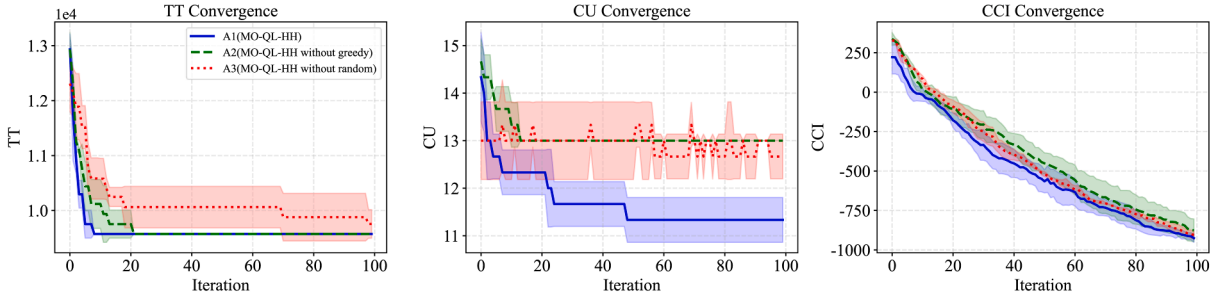
4.4. Comparison with other algorithms

To evaluate the effectiveness and superiority of the proposed algorithm, MO-QL-HH is compared with several well-known multi-objective optimization algorithms, namely NSGA-II(2002)[55], MOEA/D(2007)[56], NSGA-III(2013)[57], SPEA2(2001)[58], MOEA/D-AAWNs(2023)[59], and HGTSA(2023)[60]. For fairness, the comparison algorithms are configured with the same encoding and decoding methods, initialization generation strategy, and heuristic operators as MO-QL-HH. Each algorithm is run independently 10 times for each instance, with 100 iterations per run. For all the compared algorithms, the relevant parameters are set according to their original publications, and the population size is consistent with that of MO-QL-HH. Table 8 shows the parameters of all comparison algorithms.

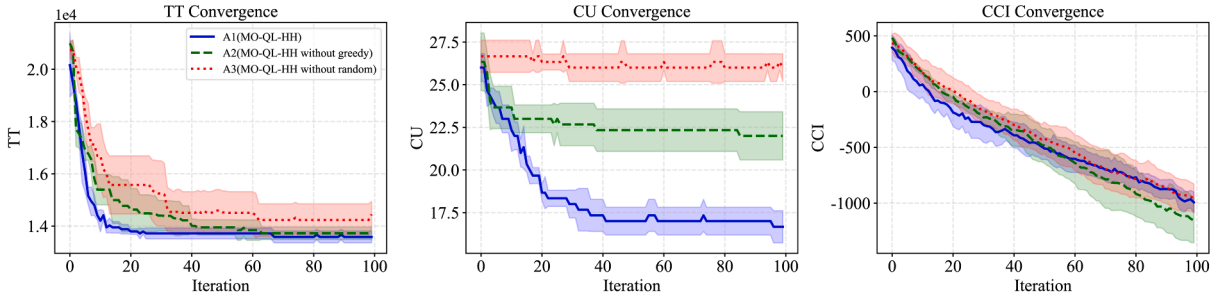
The best values for each test set are highlighted in bold.

Table 9 presents the mean values and standard deviations of HV values for all algorithms across all instances. It can be observed that among these 20 instances, compared with other algorithms, MO-QL-HH achieved the optimal average value in all Instances except Instance 17. Table 10 shows the average values and standard deviations of the IGD values of all algorithms across all instances. It can be clearly seen that, compared with other algorithms, MO-QL-HH achieved the optimal average value in all instances except instances 16, 17, and 20. Table 11 presents the C-metric comparison results of all algorithms across all instances. The results show that the experimental results of MO-QL-HH outperform HGTSA in 95 % of the instances; and outperform NSGA-II, MOEA/D, NSGA-III, SPEA2, and MOEA/D-AAWNs in all instances. Table 12 presents the average CPU running time of all algorithms across all instances, where NSGA-II yields the best result, and MO-QL-HH ranks fifth.

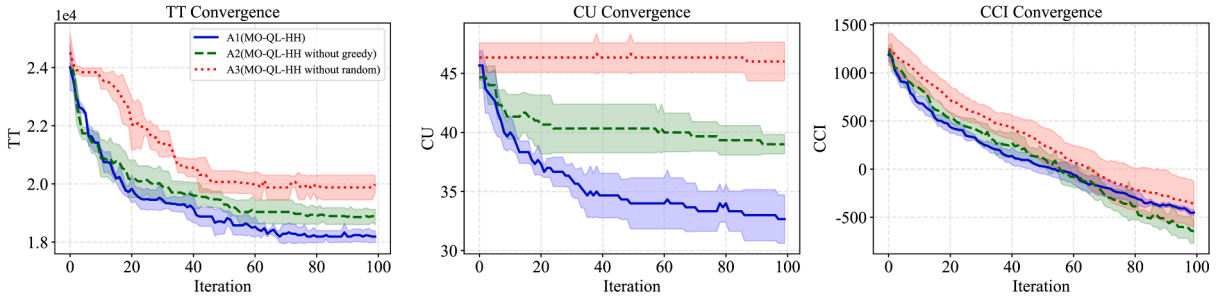
Figs. 8 and 9 show comparisons of 3D and 2D Pareto frontiers, respectively, for non-dominated solution sets from all algorithms across



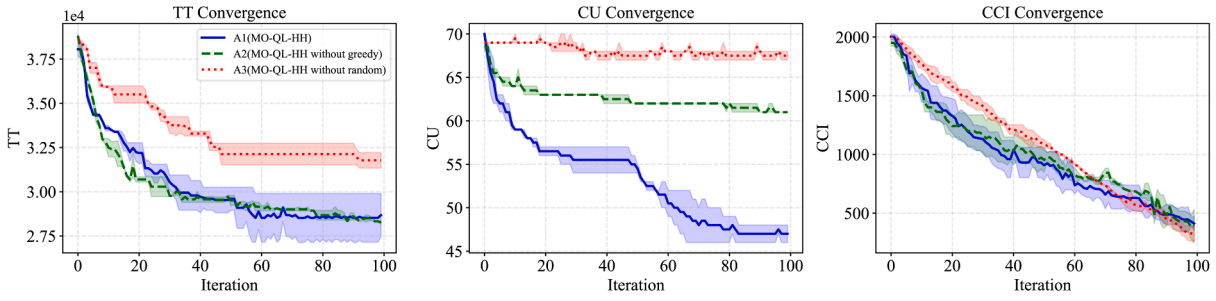
(a) Objective Convergence Analysis for Instance 1



(b) Objective Convergence Analysis for Instance 6



(c) Objective Convergence Analysis for Instance 11



(d) Objective Convergence Analysis for Instance 16

Fig. 11. The iteration graphs of MO-QL-HH and its two variants for each objective on instances of different scales.

four instances of varying scales. From Figs. 8 and 9, it can be seen that the solution set obtained by MO-QL-HH is closer to the global optimal frontier, exhibits stronger multi-objective dominance, and achieves more balanced synergistic optimization among objectives. In the comprehensive trade-off of the three objectives (*TT*, *CU*, and *CCI*), it outperforms other algorithms. Fig. 10 illustrates the visualized scheduling result of MO-QL-HH on Instance 5.

4.5. Ablation experiments on heuristic operators

To verify the effectiveness and superiority of the combination of random operators and greedy operators in the proposed algorithm, comparative experiments were conducted on all instances. The proposed algorithm was compared with two variant algorithms, namely the algorithm incorporating all operators (A1), the algorithm without greedy operators (A2), and the algorithm without random operators (A3). Each experiment was set with 100 iterations and independently run 10 times, with parameter settings consistent with those described above. Table 13 presents the comparative results of HV values and IGD values across all instances, from which it is evident that A1 achieved the optimal average values in the performance results of all instances. Table 14 presents the comparative results of C-metric and average CPU runtime across all

instances. It can be observed that, in comparison to A2, the C-metric results of A1 are superior in 85 % of the instances; relative to A3, the C-metric results of A1 outperform in all instances. Additionally, the overall average CPU runtime of A1 across all instances is lower than that of both A2 and A3.

To evaluate the convergence performance of different operator combinations on individual objectives, we selected four instances of varying scales (Instance 1, 6, 11, and 16) and plotted their average convergence curves in Fig. 11. The results clearly show that compared with the variants using incomplete operator sets, the complete operator combination (MO-QL-HH) achieves faster and more stable convergence and obtains the optimal values in most objectives across instances of different scales. This indicates that the complete low-level heuristic operator combination possesses superior convergence performance. Therefore, it can be concluded that the proposed complete low-level heuristic operator combination yields better convergence performance in solving the MCUCSP compared to individual operator combinations.

5. Conclusion and future work

This study addresses the multi-campus University Course Scheduling Problem by developing a multi-objective optimization model MCUCSP and a Q-learning-based multi-objective hyper-heuristic algorithm MO-QL-HH. The MCUCSP innovatively incorporates cross-campus teacher commuting time, classroom usage, and a comprehensive class hour index as optimization objectives, while considering both hard and soft constraints specific to multi-campus scenarios. MO-QL-HH, integrating high-level Q-learning and ten low-level heuristics, balances exploration and exploitation, supported by efficient initial solution generation and encoding-decoding mechanisms. Experimental results on 20 multi-campus instances demonstrate that MO-QL-HH outperforms well-known algorithms (NSGA-II, MOEA/D, NSGA-III, SPEA2, MOEA/D-AAWNs, and HGTSA) in HV, IGD, and C-metrics, validating its superiority in convergence and diversity. Ablation experiments verify the necessity of integrating random and greedy operators, whereas parameter optimization based on full factorial design further enhances the algorithm's performance.

While the proposed MCUCSP model and MO-QL-HH algorithm demonstrate favorable performance within the experimental multi-campus settings, several inherent limitations should be acknowledged. Firstly, the current model focuses on a static and deterministic scheduling environment, and does not accommodate dynamic disruptions such as unexpected teacher absences or classroom unavailabilities. Secondly, as the number of campuses and course tasks increases further, the algorithm may face growing challenges in terms of solving efficiency and resource consumption. Future work will focus on the following directions: extending the model to handle dynamic disruptions (e.g., teacher absences or classroom unavailability) through real-time rescheduling strategies such as rolling-horizon optimization; incorporating additional practical constraints such as student commute time and course prerequisites to enhance realism; exploring hybridizations with deep reinforcement learning to approximate Q-values in larger state spaces; and validating the algorithm on larger-scale instances while adapting it to other timetabling domains (e.g., exam scheduling or nurse rostering) for broader applicability.

Ethical approval

This article does not contain any studies with human participants or animals performed by any of the authors.

Funding

This work was supported by National Natural Science Foundation of China under Grant U21A20464, 62066005.

CRediT authorship contribution statement

Zizhuang Zhu: Writing – original draft, Methodology. **Yongquan Zhou:** Writing – review & editing, Investigation. **Guo Zhou:** Validation, Investigation. **Qifang Luo:** Writing – review & editing, Supervision. **Huajuan Huang:** Validation, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the National Natural Science Foundation of China.

Data availability

No data was used for the research described in the article.

References

[1] T. Song, M. Chen, Y. Xu, D. Wang, X. Song, X. Tang, Competition-guided multi-neighborhood local search algorithm for the university course timetabling problem, Appl. Soft. Comput. 110 (2021) 107624.
[2] A. Bashab, A.O. Ibrahim, E.E. AbedElgabar, M.A. Ismail, A. Elsafi, A. Ahmed, A. Abraham, A systematic mapping study on solving university timetabling problems using meta-heuristic algorithms, Neural. Comput. Appl. 32 (23) (2020) 17397–17432.
[3] O. Abou Kasm, B. Mohandes, A. Diabat, S. El Khatib, Exam timetabling with allowable conflicts within a time window, Comput. Ind. Eng. 127 (2019) 263–273.
[4] R. Bai, E.K. Burke, G. Kendall, J. Li, B. McCollum, A hybrid evolutionary approach to the nurse rostering problem, IEEE Trans. Evol. Comput. 14 (4) (2010) 580–590.
[5] I. Rahimi, A.H Gandomi, A comprehensive review and analysis of operating room and surgery scheduling: I. Rahimi, AH Gandomi, Arch. Comput. Methods Eng. 28 (3) (2021) 1667–1688.
[6] F. Zhao, S. Di, L. Wang, A hyperheuristic with Q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem, IEEE Trans. Cybern. 53 (5) (2022) 3337–3350.
[7] C. Artigues, M. Gendreau, L.M. Rousseau, A. Vergnaud, Solving an integrated employee timetabling and job-shop scheduling problem via hybrid branch-and-bound, Comput. Oper. Res. 36 (8) (2009) 2330–2340.
[8] L. Zhang, D. He, Y. He, B. Liu, Y. Chen, S. Shan, Real-time energy saving optimization method for urban rail transit train timetable under delay condition, Energy 258 (2022) 124853.
[9] Y. Chen, B. Mao, Y. Bai, T.K. Ho, Z. Li, Timetable synchronization of last trains for urban rail networks with maximum accessibility, Transp. Res. C: Emerg. Technol. 99 (2019) 110–129.
[10] J. Tang, Y. Yang, W. Hao, F. Liu, Y. Wang, A data-driven timetable optimization of urban bus line based on multi-objective genetic algorithm, IEEE Trans. Intell. Transp. Syst. 22 (4) (2020) 2417–2429.
[11] Z.L. Dong, C.C. Ribeiro, F. Xu, A. Zamora, Y. Ma, K. Jing, Dynamic scheduling of e-sports tournaments, Transp. Res. E: Logist. Transp. Rev. 169 (2023) 102988.
[12] D. Van Bulck, D. Goossens, J.P. Clarner, A. Dimitsas, G.H. Fonseca, C., … Lamas-Fernandez, R.M. Rosati, Which algorithm to select in sports timetabling? Eur. J. Oper. Res. 318 (2) (2024) 575–591.
[13] D. Zhang, Y. Liu, R. M'Hallah, S.C Leung, A simulated annealing with a new neighborhood structure based algorithm for high school timetabling problems, Eur. J. Oper. Res. 203 (3) (2010) 550–558.
[14] S. Ceschia, L. Di Gaspero, A. Schaerf, Educational timetabling: problems, benchmarks, and state-of-the-art results, Eur. J. Oper. Res. 308 (1) (2023) 1–18.
[15] S. Daskalaki, T. Birbas, E. Housos, An integer programming formulation for a case study in university timetabling, Eur. J. Oper. Res. 153 (1) (2004) 117–135.
[16] A.W. Siddiqui, S.A. Raza, A general ontological timetabling-model driven metaheuristics approach based on elite solutions, Expert Syst. Appl. 170 (2021) 114268.
[17] E. Goode, T. Roche, E. Wilson, J.W. McKenzie, Implications of immersive scheduling for student achievement and feedback, Stud. High. Educ. 48 (7) (2023) 1123–1136.
[18] S.C. Yeo, C.K. Lai, J. Tan, S. Lim, Y. Chandramoghan, T.K. Tan, J.J. Gooley, Early morning university classes are associated with impaired sleep and academic performance, Nat. Hum. Behav. 7 (4) (2023) 502–514.
[19] S. Daskalaki, T. Birbas, Efficient solutions for a university timetabling problem through integer programming, Eur. J. Oper. Res. 160 (1) (2005) 106–120.
[20] E.K. Burke, J. Mareček, A.J. Parkes, H. Rudová, A branch-and-cut procedure for the Udine course timetabling problem, Ann. Oper. Res. 194 (1) (2012) 71–87.

[21] N.C.F. Bagger, M. Sørensen, T.R. Stidsen, Benders' decomposition for curriculum-based course timetabling, Comput. Oper. Res. 91 (2018) 178–189.
[22] C.B. Mallari, J.L. San Juan, R. Li, The university coursework timetabling problem: an optimization approach to synchronizing course calendars, Comput. Ind. Eng. 184 (2023) 109561.
[23] A. Schaerf, A survey of automated timetabling, Artif. Intell. Rev. 13 (1999) 87–127.
[24] S.A. MirHassani, F. Habibi, Solution approaches to the course timetabling problem, Artif. Intell. Rev. 39 (2) (2013) 133–149.
[25] K. Xiang, X. Hu, M. Yu, X. Wang, Exact and heuristic methods for a university course scheduling problem, Expert Syst. Appl. 248 (2024) 123383.
[26] S.I. Hossain, M.A.H. Akhand, M.I.R. Shuvo, N. Siddique, H. Adeli, Optimization of university course scheduling problem using particle swarm optimization with selective search, Expert Syst. Appl. 127 (2019) 9–24.
[27] T. Thepphakorn, P. Pongcharoen, Performance improvement strategies on Cuckoo Search algorithms for solving the university course timetabling problem, Expert Syst. Appl. 161 (2020) 113732.
[28] T. Thepphakorn, P. Pongcharoen, Modified and hybridised bi-objective firefly algorithms for university course scheduling, Soft Comput. 27 (14) (2023) 9735–9772.
[29] J.H. Drake, A. Kheiri, E. Özcan, E.K. Burke, Recent advances in selection hyper-heuristics, Eur. J. Oper. Res. 285 (2) (2020) 405–428.
[30] J. Lin, Backtracking search based hyper-heuristic for the flexible job-shop scheduling problem with fuzzy processing time, Eng. Appl. Artif. Intell. 77 (2019) 186–196.
[31] I. Chana, Bacterial foraging based hyper-heuristic for resource scheduling in grid computing, Future Gener. Comput. Syst. 29 (3) (2013) 751–762.
[32] A. Turky, N.R. Sabar, S. Dunstall, A. Song, Hyper-heuristic local search for combinatorial optimisation problems, Knowl. Based Syst. 205 (2020) 106264.
[33] J.A. Soria-Alcaraz, E. Özcan, J. Swan, G. Kendall, M. Carpio, Iterated local search using an add and delete hyper-heuristic for university course timetabling, Appl. Soft Comput. 40 (2016) 581–593.
[34] A. Muklason, R.G. Irianti, A. Marom, Automated course timetabling optimization using tabu-variable neighborhood search based hyper-heuristic algorithm, Procedia Comput. Sci. 161 (2019) 656–664.
[35] M. Kalender, A. Kheiri, E. Özcan, E.K. Burke, A greedy gradient-simulated annealing hyper-heuristic for a curriculum-based course timetabling problem, in: 2012 12th UK workshop on computational intelligence (UKCI), IEEE, 2012, pp. 1–8.
[36] J.A. Soria-Alcaraz, G. Ochoa, J. Swan, M. Carpio, H. Puga, E.K. Burke, Effective learning hyper-heuristics for the course timetabling problem, Eur. J. Oper. Res. 238 (1) (2014) 77–86.
[37] Z.Q. Zhang, B. Qian, R. Hu, J.B. Yang, Q-learning-based hyper-heuristic evolutionary algorithm for the distributed assembly blocking flowshop scheduling problem, Appl. Soft Comput. 146 (2023) 110695.
[38] Q. Zhu, X. Wu, Q. Lin, L. Ma, J. Li, Z. Ming, J. Chen, A survey on evolutionary reinforcement learning algorithms, Neurocomputing 556 (2023) 126628.
[39] F. Zhao, Z. Geng, J. Zhang, T. Xu, A Q-learning-based multi-objective hyper-heuristic algorithm with fuzzy policy decision technology, Expert Syst. Appl. 277 (2025) 127232.
[40] S. Abdipoor, R. Yaakob, S.L. Goh, S. Abdullah, Meta-heuristic approaches for the university course timetabling problem, Intell. Syst. Appl. 19 (2023) 200253.
[41] H. Babaei, J. Karimpour, A. Hadidi, A survey of approaches for university course timetabling problem, Comput. Ind. Eng. 86 (2015) 43–59.
[42] M. Sánchez, J.M. Cruz-Duarte, J. carlos Ortíz-Bayliss, H. Ceballos, H. Terashima-Marin, I. Amaya, A systematic review of hyper-heuristics on combinatorial optimization problems, IEEE Access 8 (2020) 128068–128095.
[43] D.F. Shiau, A hybrid particle swarm optimization for a university course scheduling problem with flexible preferences, Expert Syst. Appl. 38 (1) (2011) 235–248.
[44] T. Song, S. Liu, X. Tang, X. Peng, M. Chen, An iterated local search algorithm for the University Course Timetabling Problem, Appl. Soft. Comput. 68 (2018) 597–608.
[45] R.P. Badoni, D.K. Gupta, P. Mishra, A new hybrid algorithm for university course timetabling problem using events based on groupings of students, Comput. Ind. Eng. 78 (2014) 12–25.
[46] A. Gülcü, C. Akkan, Robust university course timetabling problem subject to single and multiple disruptions, Eur. J. Oper. Res. 283 (2) (2020) 630–646.
[47] J.K. Hao, U. Benlic, Lower bounds for the ITC-2007 curriculum-based course timetabling problem, Eur. J. Oper. Res. 212 (3) (2011) 464–472.
[48] Z. Lü, J.K. Hao, Adaptive tabu search for course timetabling, Eur. J. Oper. Res. 200 (1) (2010) 235–244.
[49] P. Pongcharoen, W. Promtet, P. Yenradee, C. Hicks, Stochastic optimisation timetabling tool for university course scheduling, Int. J. Prod. Econ. 112 (2) (2008) 903–918.
[50] C.W. Fong, H. Asmuni, B. McCollum, A hybrid swarm-based approach to university timetabling, IEEE Trans. Evol. Comput. 19 (6) (2015) 870–884.
[51] X. Han, D. Wang, Gradual optimization of university course scheduling problem using genetic algorithm and dynamic programming, Algorithms 18 (3) (2025) 158.
[52] E.A. Abdelhalim, G.A. El Khayat, A utilization-based genetic algorithm for solving the university timetabling problem (uga), Alex. Eng. J. 55 (2) (2016) 1395–1409.
[53] R. Bellio, S. Ceschia, L. Di Gaspero, A. Schaerf, T. Urli, Feature-based tuning of simulated annealing applied to the curriculum-based course timetabling problem, Comput. Oper. Res. 65 (2016) 83–92.
[54] X. Guo, Q.-K. Pan, W. Zhang, Z.-H. Miao, X.-L. Jing, H.-Y. Sang, A Q-learning-assisted memetic algorithm for joint vehicle scheduling problem for harvesting and transportation in smart agriculture, Swarm Evol. Comput. 96 (2025) 102007.
[55] K. Deb, A. Pratap, S. Agarwal, T.A.M.T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Evol. Comput. 6 (2) (2002) 182–197.
[56] Q. Zhang, H. Li, MOEA/D: a multiobjective evolutionary algorithm based on decomposition, IEEE Trans. Evol. Comput. 11 (6) (2007) 712–731.
[57] K. Deb, H. Jain, An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints, IEEE Trans. Evol. Comput. 18 (4) (2013) 577–601.
[58] E. Zitzler, M. Laumanns, L. Thiele, SPEA2: improving the strength Pareto evolutionary algorithm, TIK Rep. (2001) 103.
[59] Q. Zhao, Y. Guo, X. Yao, D. Gong, Decomposition-based multiobjective optimization algorithms with adaptively adjusting weight vectors and neighborhoods, IEEE Trans. Evol. Comput. 27 (5) (2023) 1485–1497.
[60] J. Xie, X. Li, L. Gao, L. Gui, A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems, J. Manuf. Syst. 71 (2023) 82–94.
[61] Z.Q. Zhang, X.H. Qiu, B. Qian, R. Hu, L. Wang, J.B. Yang, Knowledge-enhanced multidimensional estimation of distribution hyper-heuristic evolutionary algorithm for semiconductor final testing scheduling problem, Expert Syst. Appl. 260 (2025) 125424.
[62] Z.Q. Zhang, X.Y. Wu, B. Qian, R. Hu, J.B. Yang, A Q-learning-based multi-objective hyper-heuristic algorithm for energy-efficient integrated distributed hybrid flow-shop scheduling with preventive maintenance, Comput. Oper. Res. (2025) 107267.
[63] Z.Q. Zhang, F.C. Wu, B. Qian, R. Hu, L. Wang, H.P. Jin, A Q-learning-based hyper-heuristic evolutionary algorithm for the distributed flexible job-shop scheduling problem with crane transportation, Expert Syst. Appl. 234 (2023) 121050.